

Docs5 arxitekturası və Linux/Docker əl kitabçası

by Nurkhan Masimzada

Contents

Bölmə 1. Docs5 arxitekturasını başa düşmək	15
1.1 Docs5 ekosistemi nədir	15
1.2 Əsas komponentlər	16
Web	16
PostgreSQL	16
Storage	16
Keycloak	17
Vault (və ya vault.settings.json)	17
Nginx	17
Əlavə servislər (util-lər)	17
1.3 Sorğunun sistem daxilində hərəkəti	18
1.4 Hansı problem hansı komponentdə axtarılmalıdır	20
1.5 Domen, port, konteyner və servis əlaqəsi	20
1.6 Test və production yerləşdirmə modelləri	21
Bölmə 2. Linux əsasları	22
2.1 SSH ilə qoşulma	22

2.2 Fayl sistemi	23
2.3 Navigation (pwd, ls, cd)	23
2.4 Fayl əməliyyatları (cp, mv, rm)	24
2.5 Fayl baxış və edit (cat, nano, tail)	24
2.6 Permissions (chmod, chown)	25
2.7 sudo və root	31
2.8 apt və dnf paket menecerləri ilə işləmək	31
Bölmə 3. SSH ilə serverə qoşulmaq	33
3.1 SSH nədir	33
3.2 Login/password ilə qoşulma	33
3.3 SSH key ilə qoşulma	34
3.4 Jump server üzərindən qoşulma	34
3.5 Hostname və IP ilə qoşulma fərqi	35
3.6 Qoşulma zamanı tipik xətilər	35
connection refused	36
permission denied (publickey,password)	36
no route to host	37
host key verification failed	37
3.7 SSH session təhlükəsizliyi	38
3.8 Session-dan düzgün çıxmaq	38
Bölmə 4. WinSCP ilə SFTP/SSH bağlantısı	39
4.1 WinSCP nədir	39
4.2 SFTP və SCP fərqi	40
4.3 WinSCP ilə yeni connection yaratmaq	40
4.4 Hostname, port, username, password doldurulması	40

4.5 SSH key ilə WinSCP bağlantısı	41
4.6 Saved session yaratmaq	41
4.7 Server fayl sistemində gəzmək	41
4.8 Windows-dan Linux-a fayl upload etmək	42
4.9 Linux-dan Windows-a fayl download etmək	42
4.10 Drag-and-drop və manual copy	43
4.11 Permissions və overwrite riskləri	43
4.12 WinSCP ilə uzaq faylı edit etmək	44
4.13 WinSCP-də tipik xətalər və həllər	44
Bölmə 5. Linux daxilində hərəkət və əsas komandalar	46
5.1 pwd	46
5.2 ls, ls -la	46
5.3 cd	47
5.4 tree	47
5.5 clear	48
5.6 history	48
5.7 Tab completion	48
5.8 Wildcard istifadəsi	49
5.9 Hidden file anlayışı	49
5.10 Path-lə rahat işləmək	50
Absolute path	50
Relative path	50
“Current directory” (.) və “parent” (..)	50
Bölmə 6. Fayllarla və qovluqlarla işləmək	51
6.1 mkdir	51

6.2 touch	51
6.3 cp	52
6.4 mv	52
6.5 rm	52
6.6 rm -rf riskləri	53
6.7 duplicate/backup copy yaratmaq	53
6.8 recursive copy	53
6.9 qovluq strukturunu köçürmək	54
6.10 fayl axtarmaq	54
find	54
locate	55
6.11 fayl içində mətn axtarmaq	55
grep	55
grep -r	55
6.12 fayl ölçüsü və tarixçəsini görmək	55
Bölmə 7. Fayla baxmaq və redaktə etmək	57
7.1 cat	57
7.2 less	58
7.3 more	58
7.4 head	58
7.5 tail	58
7.6 tail -f ilə log izləmək	59
7.7 nano ilə redaktə	59
7.8 vim ilə baza səviyyəsində iş	59
7.9 config faylını dəyişəndə nəzarət addımları	60

7.10 dəyişiklikdən əvvəl backup almaq	60
Bölmə 8. İcazələr və istifadəçilər	62
8.1 user, group, owner anlayışı	62
8.2 chmod	63
8.3 chown	63
8.4 executable permission	63
8.5 777 niyə təhlükəlidir	63
8.6 755, 644 tipik halları	64
8.7 permission denied xətasının analizi	64
8.8 sudo və sudo su	65
8.9 root session ilə işləmə intizamı	65
Bölmə 9. Sistem resursları və monitoring	66
9.1 top	66
9.2 htop	66
9.3 free -m	67
9.4 df -h	67
9.5 du -sh	67
9.6 uptime	68
9.7 ps aux	68
9.8 kill və kill -9	68
9.9 CPU, RAM, disk bottleneck necə anlaşılır	69
9.10 disk dolması və sistemə təsiri	69
Bölmə 10. Linux şəbəkə əsasları	71
10.1 IP, port, hostname	71
10.2 private subnet nədir	71

10.3 eyni subnetdə iki server necə danışır	72
10.4 ping	72
10.5 curl	73
10.6 ss -tuln	73
10.7 traceroute və route anlayışı	73
10.8 local və remote bağlantı məntiqi	74
Bölmə 11. Firewall və UFW	75
11.1 firewall nədir	75
11.2 UFW nədir	76
11.3 ufw status	76
11.4 ufw enable / disable	76
11.5 port açmaq	76
11.6 source IP üzrə icazə vermək	77
11.7 qaydanı silmək	77
11.8 default policy	77
11.9 deny vs allow	77
11.10 firewall problemi necə anlaşılır	78
11.11 servis açıqdır, amma port niyə görünmür	78
11.12 təhlükəsiz qayda yazma prinsipləri	79
Bölmə 12. Arxivləmə və sıxma əməliyyatları	80
12.1 tar nədir	80
12.2 tar.gz nədir	80
12.3 zip və tar fərqi	81
12.4 qovluğu tar etmək	81
12.5 tar.gz ilə compress etmək	81

12.6 archive açmaq	82
12.7 list məzmunu görmək	82
12.8 backup üçün arxivləmə ssenariləri	82
12.9 böyük qovluqları köçürmədən əvvəl yığmaq	82
12.10 Windows-a ötürmək üçün arxiv hazırlamaq	83
Bölmə 13. Linux-dan Linux-a fayl ötürmə	84
13.1 scp nədir	84
13.2 bir serverdən digərinə scp ilə fayl atmaq	84
13.3 qovluq ötürmək	85
13.4 rsync giriş səviyyəsi	85
13.5 eyni subnetdə sürətli müvəqqəti transfer yanaşması	86
13.6 SSH üzərindən transfer riskləri	86
Bölmə 14. Linux-dan Windows-a və Windows-dan Linux-a fayl ötürmə	87
14.1 Windows-da scp istifadəsi	87
14.2 PowerShell üzərindən scp	88
14.3 Linux serverdən Windows-a fayl endirmək	88
14.4 Windows-dan Linux serverə fayl atmaq	88
14.5 path format fərqləri	88
14.6 tipik scp xətalrı	89
14.7 böyük fayllar üçün tövsiyələr	89
Bölmə 15. Müvəqqəti fayl paylaşımı: Python HTTP server + wget	90
15.1 Bu yanaşma nə vaxt lazımdır	91
15.2 Python ilə qovluğu host etmək	91
15.3 hansı portda publish etmək	91

15.4 digər serverdən wget ilə çəkmək	91
15.5 lokal IP ilə fayl ötürmə ssenarisi	92
15.6 firewall və port yoxlaması	92
15.7 directory listing riskləri	92
15.8 iş bitəndən sonra host-un bağlanması	93
15.9 wget ilə tək fayl çəkmək	93
15.10 wget ilə recursive çəkmə əsasları	93
Bölmə 16. wget və curl ilə fayl və endpoint işi	94
16.1 wget nədir	94
16.2 curl və wget fərqi	94
16.3 URL-dən fayl endirmək	95
16.4 output file təyin etmək	95
16.5 self-signed / internal certificate halları	95
16.6 auth tələb edən endpoint-lər	95
16.7 connection troubleshoot	96
Bölmə 17. Docker əsasları	97
17.1 Docker nədir	97
17.2 image, container, volume, network anlayışı	98
17.3 container və VM fərqi	98
17.4 Docs5 niyə docker üzərində qurulur	98
17.5 docker version və docker info yoxlaması	99
17.6 docker ps	99
17.7 docker ps -a	99
17.8 docker images	100
17.9 container lifecycle	100

17.10 docker inspect	100
17.11 docker stats	101
Bölmə 18. Docker Compose ilə işləmək	102
18.1 docker compose nədir	102
18.2 docker-compose və docker compose fərqi	102
18.3 compose file strukturu	103
18.4 services	103
18.5 ports	103
18.6 environment	104
18.7 volumes	104
18.8 networks	104
18.9 docker compose up -d	104
18.10 docker compose down	104
18.11 docker compose stop	105
18.12 docker compose restart	105
18.13 docker compose logs -f	105
18.14 docker compose ps	106
18.15 config dəyişikliyindən sonra nə etmək	106
18.16 rebuild halları	106
18.17 orphan container və digər tipik problemlər	106
Repo-dakı util compose nümunəsi	107
Bölmə 19. Container daxilində işləmək	108
19.1 docker exec -it	108
19.2 bash və sh fərqi	109
19.3 container daxilində fayl tapmaq	109

19.4 container daxilində config oxumaq	109
19.5 container daxilində log faylı axtarmaq	109
19.6 container daxilində command işlətmək	109
19.7 container içərisinə fayl yerləşdirmək (docker cp)	110
19.8 container-dən host-a fayl çıxarmaq	110
19.9 container daxilində müvəqqəti debug	110
19.10 dəyişikliklərin persistent olub-olmaması	111
Bölmə 20. Container logları və troubleshooting	112
20.1 docker logs	112
20.2 docker logs -f	112
20.3 son sətirləri görmək	113
20.4 restart loop-ları anlamaq	113
20.5 exit code anlayışı	113
20.6 “Restarting (139)” kimi halların mənası	114
20.7 logdan root cause tapmaq	114
20.8 app log vs proxy log vs system log	114
20.9 container daxilində müvəqqəti debug	115
20.10 dəyişikliklərin persistent olub-olmaması	115
Bölmə 21. Docs5 qovluq və servis strukturu	117
21.1 /opt/wss/docs	117
21.2 /opt/wss/storage	118
21.3 /opt/wss/vault	119
21.4 /opt/wss/keycloak	119
21.5 /opt/wss/postgres və ya system PostgreSQL	119
21.6 nginx config yerləri	120

21.7 env və vault.settings faylları	120
21.8 hansı servis hansı domainə cavab verir	121
Bölmə 22. Nginx və reverse proxy əsasları	122
22.1 reverse proxy nədir	122
22.2 upstream nədir	123
22.3 proxy_pass	123
22.4 80 və 443 məntiqi	123
22.5 SSL certificate və private key	124
22.6 server_name	124
22.7 nginx config test	124
22.8 common 502 səbəbləri	125
22.9 upstream connection refused	125
22.10 nginx log oxuma əsasları	126
Bölmə 23. PostgreSQL-ə qoşulma: pgAdmin və digər üsullar	127
23.1 PostgreSQL connection anlayışı	127
23.2 host, port, db name, username, password	128
23.3 pgAdmin nədir	128
23.4 pgAdmin ilə yeni server connection yaratmaq	128
23.5 pgAdmin-də General və Connection hissələri	129
23.6 SSH tunnel ilə qoşulma məntiqi	129
23.7 Nə zaman SSH tunnel lazımdır	129
23.8 pgAdmin içində SSH tunneling konfigurasiyası	129
23.9 local port forwarding məntiqi	130
23.10 connection failure halları	130
23.11 psql ilə alternativ qoşulma	130

23.12 DB list və table list yoxlaması	131
23.13 readonly işləmək intizamı	131
Bölmə 24. PostgreSQL ilə baza səviyyəsində işləmək	132
24.1 \l	132
24.2 \c	132
24.3 \dt	133
24.4 basic select	133
24.5 LIMIT ilə təhlükəsiz yoxlama	133
24.6 update/delete riskləri	133
24.7 migration zamanı DB-lərin yoxlanması	134
24.8 owner və permission anlayışı	134
24.9 connection refusal, pg_hba, listen_addresses əsasları	135
Bölmə 25. Postman ilə API yoxlamaları	136
25.1 Postman nədir	136
25.2 GET request atmaq	137
25.3 POST request atmaq	137
25.4 Headers əlavə etmək	137
25.5 X-API-KEY nədir	137
25.6 X-API-KEY ilə request	138
25.7 Authorization header anlayışı	138
25.8 query param və path param	138
25.9 body göndərmək	139
25.10 response oxumaq	139
25.11 SSL verify problemləri	139
25.12 internal API endpoint-lərin test edilməsi	140

25.13 curl-dan Postman-a və Postman-dan curl-a keçid	140
Bölmə 26. Vault, Keycloak və auth əsasları	141
26.1 Docs5-də auth axını	141
26.2 Keycloak nə üçündür	142
26.3 Vault nə üçündür	142
26.4 BA nə qədər bilməlidir	143
26.5 connection probleminə ilkin yoxlama	143
26.6 domain, port, TLS, credentials əlaqəsi	143
26.7 bu servislər down olanda nə baş verir	144
Bölmə 27. Storage əməliyyat məntiqi	145
27.1 WSS Storage nədir	145
27.2 blob anlayışı	146
27.3 storage metadata və blob data fərqi	146
27.4 storage servisin ayaqda olub-olmadığını yoxlamaq	146
27.5 storage config faylları	146
27.6 migration kontekstində storage	147
27.7 preview və file access problemləri	147
27.8 storage ilə bağlı tipik səhvlər	148
Bölmə 28. Portlar və firewall qaydaları (hansı server → hansı server → hansı port)	149
28.1 Port anlayışı: inbound vs outbound	150
28.2 Tipik Docs5 komponentləri və portları	150
28.3 Arxitektura variantları (port ehtiyacı buna görə dəyişir)	151
Variant A: “Hər şey 1 serverdə” (ən sadə)	151
Variant B: “Servislər ayrı serverlərdə” (ən real prod)	151

28.4 Tam port matrisi (serverdən serverə)	152
28.5 UFW (və ya iptables) səviyyəsində nümunə qaydalar	153
28.5.1 Nginx server (DMZ) üçün	153
28.5.2 App server üçün	154
28.5.3 DB server üçün	155
28.6 Sizin mühit üçün “tam və dəqiq” port siyahısını çıxartmaq (5 dəqiqəlik prosedur)	155
Addım 1: hər serverdə dinləyən portları çıxart	155
Addım 2: Docker konteyner port publish-lərini yoxla (əgər container-dirsə)	156
Addım 3: Nginx upstream portunu dəqiq tap	156
Addım 4: App konfiqində DB host/port-a bax	156
Addım 5: Çıxan “real portlar” ilə cədvəli yenilə	157
28.7 BA üçün praktiki qayda	157

Bölmə 1. Docs5 arxitekturasını başa düşmək

Bu bölmədə Docs5 ekosistemini “komponentlər + əlaqələr” kimi düşünməyi öyrənəcəyik. Məqsəd odur ki, problem gələndə “haradan başlayaq?” sualına sistematik cavab verə biləsiniz.

1.1 Docs5 ekosistemi nədir

Docs5 adətən bir neçə servis və asılılıqdan ibarət olur:

- istifadəçi interfeysi (web)
- API/back-end servis(lər)i
- verilənlər bazası (PostgreSQL)
- fayl saxlama (Storage)
- autentifikasiya/avtorizasiya (Keycloak)
- secret/konfiq idarəetməsi (Vault və ya vault.settings)

- reverse proxy (Nginx)
- əlavə util-lər (migration, preview generation və s.)

BA üçün əsas fikir: sistem “tək tətbiq” deyil. Bir yerdə problem görünəndə, səbəb başqa komponentdə ola bilər.

1.2 Əsas komponentlər

Web

- User browser burada işləyir.
- Tipik problem simptomları: səhifə açılmır, 502/504, login loop, JS error.

PostgreSQL

- Metadata, biznes obyektləri, sistem parametrləri, iş axınları və s. burada olur.
- Tipik simptomlar: səhifələr yavaş açılır, axtarış qırılır, “DB connection failed”, migrasiya problemləri.

Storage

- Faylların binar hissəsi (blob data) saxlanır.
- Tipik simptomlar: fayl upload/download işləmir, preview açılmır, “file not found”, “access denied”.

Keycloak

- SSO/login, token, realm/client konfiqurasiyası.
- Tipik simptomlar: login alınmır, token invalid, redirect səhv, “unauthorized”.

Vault (və ya `vault.settings.json`)

- Secret-lar: connection string, password, API key və s.
- Tipik simptomlar: servis start olmur (config çatmır), auth/DB bağlantısı qırılır.

Qeyd: Bu repo-da util-lərin `appsettings.json` faylları çox vaxt yalnız `"PrependConfigFiles": ["vault.settings.json"]` kimi sadə olur. Bu o deməkdir ki, real connection string və secret-lar ayrıca `vault.settings.json` içində saxlanılır və konteynerə mount edilir.

Nginx

- Reverse proxy: domain-ləri servis(lər)-ə yönləndirir.
- Tipik simptomlar: 502 Bad Gateway, 404/403, TLS certificate error.

Əlavə servislər (util-lər)

Docs5 əməliyyatlarında tez-tez “utility” konteynerləri işlədilir:

- DB migration
 - schema/table/column renaming
 - preview generation
-

- storage migration

Bu repodakı Sources/utils/*/docker-compose.yml fayllarında bu pattern görünür: bir “dotnet util” image-i işə düşür, öz DLL-ni və appsettings.json-u mount edir.

Misal pattern (sadələşdirilmiş):

```
version: "3.3"
services:
  wssutil:
    image: images.wss-consulting.ru/dot/docs/prod:latest
    restart: "no"
    volumes:
      - "/etc/ssl/certs:/etc/ssl/certs:ro"
      - "./appsettings.json:/app/appsettings.json:ro"
      - "../../.vault.settings.json:/app/vault.settings.json:ro"
      - "./Some.Util.dll:/app/util.dll:ro"
      - "./Some.Util.runtimeconfig.json:/app/
util.runtimeconfig.json:ro"
      - "./Some.Util.deps.json:/app/util.deps.json"
    entrypoint: ["dotnet", "util.dll"]
```

BA üçün nəticə: belə util-lər adətən “bir dəfə işləyib çıxır” (restart: no), logları oxunur, nəticəyə görə növbəti addım seçilir.

1.3 Sorğunun sistem daxilində hərəkəti

İstifadəçi browser-də <https://docs.example.az> açır:

1. DNS domeni server IP-sinə yönləndirir
2. Nginx 443 portunda sorğunu qəbul edir
3. Nginx upstream kimi web/app servisinə yönləndirir
4. App servisi ehtiyac olduqda:
 - PostgreSQL-ə query atır
 - Storage-dan fayl oxuyur
 - Keycloak-dan token yoxlayır
5. Cavab Nginx → browser geri qaydır

Mermaid sxem (sadə):

sequenceDiagram

participant U as User Browser

participant N as Nginx

participant A as Docs5 App/API

participant K as Keycloak

participant P as PostgreSQL

participant S as Storage

U->>N: HTTPS request

N->>A: proxy_pass to upstream

A->>K: token/SSO validation (optional)

A->>P: SQL queries

A->>S: read/write blobs (optional)

A-->>N: HTTP response

N-->>U: HTTPS response

1.4 Hansı problem hansı komponentdə axtarılmalıdır

Triage üçün “simptom → ehtimal olunan komponent” xəritəsi:

- 502 Bad Gateway → Əgər Docs loglarda error vermirsə deməli Nginx upstream-ə çıxıb bilmir (app down, port dəyişib, network problem)
- Login alınmır → Keycloak (realm/client), Nginx redirect, time drift
- Fayl preview açılmır → Storage, preview generation util, app config
- DB connection error → PostgreSQL servis, credential/secret, network, pg_hba
- Yavaşlıq → DB performans, disk doluluğu, CPU/RAM, network

Prinsip: Ən əvvəl “aşağı qat” yoxlanılır:

- servis ayaqdadır?
- port açıqdır?
- logda konkret error var?

1.5 Domen, port, konteyner və servis əlaqəsi

Ümumi model:

- Domain (məs: docs.company.az) → Nginx (80/443)
- Nginx server_name ilə uyğun bloku seçir
- proxy_pass ilə daxiləki upstream-ə yönləndirir:
 - ya host port (məs: 127.0.0.1:5000)
 - ya docker network servis adı (məs: http://docs-app:5000)

BA üçün kritik suallar:

- İstifadəçi hansı domain-ə girir?
- Həmin domain Nginx-də hara proxy olunur?
- Proxy olunan servis/container işləyirmi?

1.6 Test və production yerləşdirmə modelləri

Docs5 yerləşdirməsi müxtəlif ola bilər:

- Single host + Docker Compose: ən tipik; Nginx + app + db + keycloak + storage eyni hostda.
- Separated DB: PostgreSQL ayrıca serverdə; app-lar docker hostda.
- Kubernetes: bəzi təşkilatlarda.

BA üçün fərq nədir?

- “Harada axtarım?” sualının cavabı dəyişir.
- Portlar, network segmentlər, firewall qaydaları daha kritik olur.

Növbəti bölmədə Linux əsasları: fayl sistemi, permission, sudo, paket menecerləri və ən vacib gündəlik komandalar.

Bölmə 2. Linux əsasları

Bu bölmə “serverdə it-bata düşməmək” üçün minimum Linux biliklərini verir. Məqsəd DevOps olmaq deyil; məqsəd təhlükəsiz şəkildə:

- qovluğu tapmaq,
- faylı oxumaq,
- log izləmək,
- permission problemini başa düşmək,
- read-only yoxlamaları etməkdir.

2.1 SSH ilə qoşulma

Linux serverlərə adətən SSH ilə qoşuluruq.

Ən tipik format:

```
ssh username@server_ip
```

Port standartdan fərqlidirsə:

```
ssh -p 2222 username@server_ip
```

Əgər jump/bastion istifadə olunursa, bu Bölmə 3-də detallı veriləcək.

2.2 Fayl sistemi

Linux-da fayl sistemi kökdən başlayır: /.

Ən çox rast gəlinən qovluqlar:

- /home/<user> — user-in “ev” qovluğu
- /etc — sistem konfiqləri
- /var/log — log faylları
- /opt — tətbiqlər üçün tez-tez istifadə olunan yer (Docs5 kimi deploy-larda çox rast gəlinir)
- /tmp — müvəqqəti fayllar

BA üçün əsas:

- nəyisə axtaranda əvvəlcə “tətbiq harada yerləşir?” sualını verin.
- prod-da təsadüfi qovluqlarda fayl yaratmaqdan çəkinin.

2.3 Navigation (pwd, ls, cd)

- pwd — harada olduğunuzu göstərir
- ls — qovluğun içini göstərir
- cd — qovluq dəyişir

Misal:

`pwd`

`ls -la`

`cd /opt`

`ls -la` xüsusilə faydalıdır, çünki:

- gizli faylları göstərir (`.` ilə başlayan)
- owner/permission göstərir

2.4 Fayl əməliyyatları (`cp`, `mv`, `rm`)

- `cp` — kopyalama
- `mv` — köçürmə/rename
- `rm` — silmə

Təhlükəsiz yanaşma:

- dəyişiklik etməzdən əvvəl backup kopya yaradın

Misal:

`cp appsettings.json appsettings.json.bak`

Prod-da `rm -rf` “qırmızı bayraqdır”. Əmin deyilsinizsə, etməyin.

2.5 Fayl baxış və edit (`cat`, `nano`, `tail`)

Read-only baxış üçün ən çox istifadə edilənlər:

- `cat file` — faylı tam çıxarır (kiçik fayllar üçün)

- `less file` — böyük faylda rahat gəzmək (q- ilə çıxılır)
- `head -n 50 file` — ilk 50 sətir
- `tail -n 200 file` — son 200 sətir
- `tail -f file` — logu canlı izləmək

Edit üçün:

- `nano file` — sadə editor (Ctrl+O yaz, Ctrl+X çıx)

BA üçün tövsiyə: prod-da edit etmək lazımdırsa, mümkün qədər dəyişiklikləri əvvəlcə test/pre-prod-da hazırlayıb, prod-da minimal risklə tətbiq etmək daha düzgündür.

2.6 Permissions (chmod, chown)

Linux-da permission 3 rola bölünür:

- owner (u)
- group (g)
- others (o)

və 3 icazə növü var:

- read (r)
- write (w)
- execute (x)

Qısa məna:

- `read (r):`
 - faylda: məzmunu oxumağa icazə verir
 - qovluqda: içindəki adları görməyə icazə verir
- `write (w):`
 - faylda: faylı dəyişməyə icazə verir
 - qovluqda: içində fayl yaratmaq/silmək/rename etməyə icazə verir
- `execute (x):`
 - faylda: faylı proqram kimi işə salmağa icazə verir
 - qovluqda: qovluğa daxil olmağa (`cd`) və içində gəzməyə icazə verir

`ls -la` çıxışında misal:

- `-rw-r--r--` → fayl
- `drwxr-xr-x` → qovluq

Burada ilk simvol obyekt tipidir:

- `-` adi fayl
- `d` qovluq

Sonrakı 9 simvol 3-3 bölünür:

- `rw-` (owner)
- `r--` (group)
- `r--` (others)

Rəqəmsal (numeric) permission məntiqi:

- `r = 4`
- `w = 2`

- $x = 1$

Toplanaraq yazılır:

- $7 = rwx = 4+2+1$
- $6 = rw- = 4+2$
- $5 = r-x = 4+1$
- $4 = r-- = 4$

Tez-tez istifadə olunan permission kodları:

- $777 \rightarrow rwxrwxrwx$
 - owner, group, others hamısı oxu/yaz/execute edə bilir
 - praktikada çox risklidir, prod üçün tövsiyə edilmir
- $755 \rightarrow rwxr-xr-x$
 - owner tam səlahiyyətlidir, qalanlar oxu+execute
 - qovluqlar və executable script-lər üçün çox yaygındır
- $775 \rightarrow rwxrwxr-x$
 - owner və group tam səlahiyyətli, others oxu+execute
 - komanda ilə ortaq işlənən deploy qovluqlarında istifadə oluna bilər
- $700 \rightarrow rwx-----$
 - yalnız owner daxil ola və dəyişə bilər
 - private qovluqlar və həssas fayllar üçün uyğundur
- $655 \rightarrow rw-r-xr-x$
 - owner oxu+yazır, group/others oxu+execute edir
 - adi config faylları üçün çox yaygın deyil, amma bəzi xüsusi hallarda görünə bilər

- 644 → `rw-r--r--`
 - owner oxu+yazı, qalanlar yalnız oxu
 - config və text faylları üçün ən çox istifadə olunan seçimdir
- 600 → `rw-----`
 - yalnız owner oxu+yazı edə bilər
 - secret tipli fayllar üçün (məs: private key) standart seçimdir

Tipik vəziyyət:

- config faylları: 644 (`rw-r--r--`)
- qovluqlar: 755 (`rxrxr-xr-x`)

`chmod` icazəni dəyişir. Məsələn:

```
chmod 644 appsettings.json
```

Yəni:

- owner oxuya və yazı bilər
- group və others yalnız oxuya bilər

Faylı icra edilən script etmək üçün:

```
chmod +x deploy.sh
```

Bu, owner/group/others üçün execute bitini əlavə edir. Daha dəqiq vermək istəsəniz:

```
chmod u+x deploy.sh
```

Yalnız owner üçün `x` əlavə edir.

Praktik komandalar:

```
# Cari permission-a bax
```

```
ls -l appsettings.json
```

```
# Simvolik formatla icazə ver
```

```
chmod u=rw,g=r,o=r appsettings.json
```

```
# Qovluq və altındakı hər şeyə tətbiq et (ehtiyatla)
```

```
chmod -R 755 /opt/myapp/scripts
```

chown owner-i dəyişir:

```
sudo chown root:root appsettings.json
```

owner:group hissəsində ad əvəzinə rəqəm də ola bilər.

- 1000:1000 kimi yazılış UID:GID deməkdir
- birinci 1000 istifadəçinin ID-si (UID)
- ikinci 1000 qrupun ID-si (GID)

Məsələn:

```
sudo chown 1000:1000 appsettings.json
```

Bu komanda faylın sahibini UID 1000, qrupunu GID 1000 edir.

Yoxlama komandaları:

```
# Fayl sahibinin adını və qrupunu göstərir
```

```
ls -l appsettings.json
```

```
# Fayl sahibinin UID/GID rəqəmlərini göstərir
```

```
ls -ln appsettings.json
```

```
# Cari istifadəçinin UID/GID məlumatı  
id
```

Format:

```
sudo chown <owner>:<group> <fayl_veya_qovluq>
```

Məsələn, tətbiq fayllarını docs5 istifadəçisi və qrupu ilə əlaqələndirmək:

```
sudo chown -R docs5:docs5 /opt/docs5
```

Yalnız group dəyişmək üçün chgrp:

```
sudo chgrp nginx /opt/docs5/logs
```

Yoxlama üçün:

```
ls -la /opt/docs5
```

```
id
```

```
groups
```

“777 ver, keçsin” yanaşması təhlükəlidir. İcazəni həmişə minimum ehtiyac prinsipi ilə verin.

777 (rwxrwxrwx) niyə risklidir:

- hər kəs yazıb dəyişə bilər
- təsadüfi və ya zərərli dəyişiklik ehtimalı artır
- audit və təhlükəsizlik intizamı pozulur

BA üçün qayda:

- əvvəl `ls -la` ilə real vəziyyəti yoxlayın
- sonra ən dar icazəni verin
- dəyişiklikdən sonra yenidən `ls -la` ilə təsdiq edin

2.7 sudo və root

- `sudo` bir əməliyyatı admin səlahiyyəti ilə icra edir.
- `root` sistemin ən yüksək səlahiyyətli istifadəçisidir.

Prod intizamı:

- `root` ilə uzun müddət işləməyin.
- `sudo` istifadə edəndə “həqiqətən lazımdır?” sualını verin.

2.8 apt və dnf paket menecerləri ilə işləmək

Serverin OS-nə görə fərqlənir:

- Ubuntu/Debian → `apt`
- RHEL/CentOS/Fedora → `dnf` (və ya köhnə `yum`)

Məqsəd: paket qurmaq, update etmək.

Ubuntu nümunəsi:

```
sudo apt update
```

```
sudo apt install -y httpd
```


RHEL/Fedora nümunəsi:

```
sudo dnf install -y htop
```

Prod-da paket install/update dəyişiklik sayılır. Əgər prosedur tələb olunursa, change request olmadan etməyin.

Növbəti bölmə SSH bağlantısını daha detallı izah edəcək: password vs key, jump server, tipik xətilər və təhlükəsiz sessiya davranışı.

Bölmə 3. SSH ilə serverə qoşulmaq

SSH (Secure Shell) Linux serverə təhlükəsiz (şifrəli) qoşulmaq üçün standart üsuldur.

3.1 SSH nədir

SSH sizə uzaq serverdə terminal açmağa imkan verir:

- komandaları sanki serverin yanında oturmuş kimi icra edirsiniz
- trafik şifrələnir

3.2 Login/password ilə qoşulma

Format:

```
ssh username@hostname
```

Qoşulanda sizdən password istəyə bilər.

Prod tövsiyəsi:

- password-u heç vaxt chat-a yazmayın
- password manager istifadə edin
- mümkün olduqda SSH key-ə keçin

3.3 SSH key ilə qoşulma

SSH key cütükdür:

- private key — sizdə olur, paylaşılır
- public key — serverdə authorized_keys-ə əlavə olunur

Qoşulma:

```
ssh -i ~/.ssh/id_rsa username@hostname
```

Əgər key parol ilə qorunursa, passphrase soruşacaq.

3.4 Jump server üzərindən qoşulma

Bəzən prod serverlər birbaşa internetdən açıq olmur. Bu zaman əvvəlcə “jump/bastion” serverə girib, ordan hədəf serverə keçirsiniz.

Sadə iki addım:

```
ssh ba@jump.company.az
```

```
ssh appuser@10.0.10.25
```

Daha rahat “ProxyJump” ilə (lokal ~/.ssh/config):

```
Host prod-app
```

```
  HostName 10.0.10.25
```

```
  User appuser
```

```
  ProxyJump ba@jump.company.az
```

Sonra:

```
ssh prod-app
```

3.5 Hostname və IP ilə qoşulma fərqi

- IP sabit ola bilər, amma bəzən dəyişir.
- Hostname DNS ilə IP-yə çevrilir.

Troubleshooting zamanı:

- hostname işləmirsə, IP ilə yoxlayın
- DNS problemindən şübhələnilirsinizsə, nslookup/dig (əgər icazə varsa) istifadə olunur

3.6 Qoşulma zamanı tipik xətlər

Bu hissə BA üçün ən vacib hissələrdəndir: xəta mətnini oxuyub ilk səbəbi təxmin etmək.

connection refused

Mənası: serverə çatdınız, amma həmin portda SSH servisi cavab vermir.

Tipik səbəblər:

- yanlış port
- sshd down
- firewall portu bağlayıb

Nə yoxlamaq olar:

- portu dəqiqləşdirin (-p istifadə edin)
- eyni hosta ping/curl mümkünmü (şəbəkə siyasətindən asılıdır)

permission denied (publickey,password)

Mənası: server sizi qəbul etmir.

Tipik səbəblər:

- username səhv
- password səhv
- key yanlış seçilib
- sizin user-in serverdə icazəsi yoxdur

Nə etmək:

- username-i dəqiqləşdirin
- doğru key faylını seçin (-i)
- DevOps/Security-dən user icazəsini yoxlatdırın

no route to host

Mənası: şəbəkə səviyyəsində serverə yol yoxdur.

Tipik səbəblər:

- VPN qoşulu deyil
- subnet arası routing yoxdur
- firewall şəbəkə səviyyəsində bloklayır

Nə etmək:

- VPN statusunu yoxlayın
- jump server tələb olunurmu, dəqiqləşdirin

host key verification failed

Mənası: SSH serverin “fingerprint”i dəyişib və sizdəki `known_hosts` ilə uyğun gəlmir.

Səbəb ola bilər:

- server yenidən qurulub
- IP başqa serverə keçib
- (nadir) man-in-the-middle risk

Tövsiyə:

- Prod-da bu xətdə dərhal “sil keçsin” etməyin.
- DevOps-dan yeni fingerprint təsdiqi alın.

3.7 SSH session təhlükəsizliyi

BA üçün praktiki qaydalar:

- eyni anda çoxlu serverdə root sessiya açmayın
- əməliyyatları “copy-paste” edəndə iki dəfə yoxlayın
- history-də secret qalmasın deyə:
 - secret-ları komanda sətrində yazmamaq daha yaxşıdır
 - mümkün olduqda environment/secret file istifadə olunur

3.8 Session-dan düzgün çıxmaq

- sadə çıxış: `exit`
- qısa yol: `Ctrl+D`

Əgər sessiya “donubsa”:

- `~.` (tilda + nöqtə) SSH bağlantısını kəsə bilər

Növbəti bölmə WinSCP ilə fayl ötürməni və uzaq fayl redaktəsini izah edir.

Bölmə 4. WinSCP ilə SFTP/SSH bağlantısı

WinSCP Windows istifadəçiləri üçün serverə fayl atmaq/çəkmək və bəzən uzaq fayla baxmaq üçün çox istifadə olunan alətdir.

Qeyd: Prod mühitdə “fayl yüklədim” əməliyyatı da dəyişiklik sayılır. Xüsusən config fayllarında overwrite riski var.

4.1 WinSCP nədir

WinSCP:

- SFTP/SCP ilə fayl transfer edir
- remote faylları görməyə və (icazə varsa) edit etməyə imkan verir
- session-ları yadda saxlayır

4.2 SFTP və SCP fərqi

- SFTP (SSH File Transfer Protocol) — SSH üstündə fayl transfer protokolu, daha geniş funksionallıq.
- SCP — daha sadə “copy” mexanizmi.

Praktikada:

- çox vaxt SFTP seçilir
- təşkilatın siyasətinə görə məhdudiyyət ola bilər

4.3 WinSCP ilə yeni connection yaratmaq

1. WinSCP açın
2. New Site seçin
3. File protocol: adətən SFTP
4. Host name, Port number, User name daxil edin

4.4 Hostname, port, username, password doldurulması

Tipik:

- Host name: `jump.company.az` və ya `10.0.10.25`
- Port: 22 (və ya sizə verilən port)
- User name: `ba`, `appuser` və s.

Password:

- bir dəfəlik daxil etmək mümkündür
- amma təhlükəsizlik üçün key daha üstündür

4.5 SSH key ilə WinSCP bağlantısı

Əgər sizdə private key varsa:

- WinSCP-də Advanced... → SSH → Authentication
- Private key file seçin

WinSCP bəzən .ppk format istəyir (PuTTY key). OpenSSH formatını çevirmək lazım ola bilər.

4.6 Saved session yaratmaq

Eyni serverə tez-tez girirsinizsə:

- Save edin
- session adı verin: prod-app (via jump) kimi

Prod üçün tövsiyə:

- session adında mühiti yazın (test/pre-prod/prod)
- səhv mühitə upload riskini azaldır

4.7 Server fayl sistemində gəzmək

WinSCP iki panel göstərir:

- sol: lokal kompüter
- sağ: server

Serverdə tipik qovluqlar:

- /opt/... (tətbiq yerləşimi)
- /var/log/... (loglar)
- /etc/... (nginx config və s.)

4.8 Windows-dan Linux-a fayl upload etmək

Addımlar:

1. sağ paneldə hədəf qovluğa keçin
2. sol paneldən faylı seçin
3. drag-and-drop və ya Upload

Risk:

- eyni adlı fayl varsa overwrite edə bilərsiniz

Təhlükəsiz yanaşma:

- əvvəl serverdə .bak backup yaradın
- upload edəndə “Overwrite” etməzdən əvvəl iki dəfə yoxlayın

4.9 Linux-dan Windows-a fayl download etmək

Bu, log və ya backup faylları üçün lazımlı ola bilər:

1. serverdən faylı seçin
2. lokal qovluğa sürükləyin

Tövsiyə:

- böyük fayllarda (GB) download vaxt aparır
- əvvəl zip/tar.gz edib ölçünü azaldmaq olar

4.10 Drag-and-drop və manual copy

- Drag-and-drop rahatdır, amma “yanlış qovluğa atdım” riski var.
- Manual copy (Copy → Paste) daha nəzarətlidir.

Prod-da tövsiyə:

- dəyişiklik edərkən manual və daha kontrollu üsul seçin.

4.11 Permissions və overwrite riskləri

Əgər upload etdiyiniz faylın owner/permission-ı dəyişirsə:

- servis həmin faylı oxuya bilməyə bilər
- permission denied kimi error çıxar

BA üçün praktik:

- “niyə servis açılmır?” sualında son dəyişikliklərə baxın: config faylı upload olunub?

4.12 WinSCP ilə uzaq faylı edit etmək

WinSCP remote faylı edit edəndə adətən belə edir:

- faylı lokala çəkir
- edit edir
- geri yükləyir

Risqlər:

- concurrency (siz edit edərkən başqa biri də dəyişir)
- backup olmadan dəyişiklik

Prod-da minimal intizam:

- edit etməzdən əvvəl backup
- dəyişiklikləri kiçik saxlayın
- dəyişiklikdən sonra config test varsa, icra edin (məsələn, nginx config test)

4.13 WinSCP-də tipik xətlər və həllər

- Network error: Connection timed out → VPN/jump/firewall
- Authentication failed → username/password/key problem
- Permission denied (upload zamanı) → hədəf qovluğa yazmaq icazəniz yoxdur
- Host key has changed → host key verification; təsdiq olmadan davam etməyin

Növbəti bölmə Linux daxilində gündəlik komandaları daha sistemli şəkildə izah edir: `pwd`, `ls`, `cd`, `tree`, `history`, `wildcard` və s.

Bölmə 5. Linux daxilində hərəkət və əsas komandalar

Bu bölmə Linux-da “tez və təhlükəsiz” işləmək üçün gündəlik komandaları sistemli izah edir.

5.1 `pwd`

“Print working directory” — hazırda hansı qovluqdasınız.

`pwd`

Triage zamanı faydalıdır: kiməsə “mən burdayam” demək üçün də.

5.2 `ls`, `ls -la`

Qovluq içini göstərir:

ls

ls -la

-la niyə vacibdir:

- gizli fayllar görünür (. ilə başlayanlar)
- permission/owner görünür

5.3 cd

Qovluq dəyişmək:

cd /opt

cd ~

cd -

- cd ~ — home qovluğa qaytarır
- cd - — əvvəlki qovluğa qaytarır

5.4 tree

Qovluq strukturunu ağac kimi göstərir.

tree

Dərinliyi limitləmək:

tree -L 2

Əgər tree qurulu deyilsə, OS-dən asılı olaraq paket meneceri ilə quraşdırıla bilər (Bölmə 2.8).

5.5 `clear`

Terminal ekranını təmizləyir:

```
clear
```

Qısa yol: `Ctrl+L`.

5.6 `history`

Əvvəl işlədilən komandaları göstərir:

```
history
```

```
history | tail -n 20
```

Təhlükəsizlik qeydi:

- komandada secret yazmamaq daha doğrudur; `history`-də qala bilər.

5.7 Tab completion

Linux shell-də (`bash/zsh`) tab completion çox vaxt edir:

- qovluq/fayl adını avtomatik tamamlayır
- typo riskini azaldır

Misal:

- `cd /op` yazıb Tab basanda `/opt` tamamlaya bilər.

5.8 Wildcard istifadəsi

Wildcard (şablon) fayl seçməyə kömək edir:

- * — istənilən simvol ardıcılığı
- ? — tək simvol

Misal:

```
ls *.json
```

```
ls docker-compose*.yaml
```

Risk:

- wildcard ilə rm işlətmək təhlükəlidir. Silməzdən əvvəl ls ilə yoxlayın.

5.9 Hidden file anlayışı

. ilə başlayan fayllar gizlidir.

Misal:

- .env
- .bashrc
- .ssh/

Görmək üçün:

```
ls -la
```

5.10 Path-lə rahat işləmək

BA üçün ən çox qarışan mövzular:

- absolute path vs relative path

Absolute path

/ ilə başlayır və harada olduğunuzdan asılı deyil:

```
cat /etc/hosts
```

Relative path

Hazırkı qovluğa görə işləyir:

```
cat ./appsettings.json
```

```
cat ../vault.settings.json
```

“Current directory” (.) və “parent” (..)

- . → cari qovluq
- .. → bir üst qovluq

Misal:

```
ls .
```

```
ls ..
```

Növbəti bölmələrdə fayllarla iş (mkdir, cp, mv, rm), find/grep ilə axtarış və config dəyişəndə nəzarət addımları daha geniş izah olunacaq.

Bölmə 6. Fayllarla və qovluqlarla işləmək

Bu bölmədə serverdə fayl/qovluq əməliyyatlarını təhlükəsiz icra etməyi öyrənəcəksiniz. Məqsəd: “kopyaladım, itdi” tipli riskləri azaltmaq.

6.1 `mkdir`

Qovluq yaratmaq:

```
mkdir logs
```

```
mkdir -p /tmp/docs5/debug
```

- `-p` ara qovluqları da yaradır.

6.2 `touch`

Boş fayl yaratmaq və ya mövcud faylın timestamp-ini yeniləmək:

```
touch notes.txt
```

6.3 cp

Fayl kopyalama:

```
cp appsettings.json appsettings.json.bak
```

Qovluq kopyalama (recursive):

```
cp -r /opt/wss/docs /opt/wss/docs.bak
```

6.4 mv

Köçürmə və ya rename:

```
mv appsettings.json appsettings.json.old
```

```
mv file.txt /tmp/
```

6.5 rm

Silmə:

```
rm file.txt
```

Qovluq silmək üçün (daxili ilə birlikdə):

```
rm -r folder
```

6.6 `rm -rf` riskləri

`rm -rf`:

- soruşmadan silir (-f)
- qovluğu içi ilə silir (-r)

Prod-da bu əməliyyat ən riskli əmərlərdəndir.

Minimal intizam:

1. Əvvəl `ls` ilə nə sildiyini gör
2. Wildcard varsa, daha da diqqətli ol
3. Əmin deyilsənsə, etmə

6.7 duplicate/backup copy yaratmaq

Config dəyişməzdən əvvəl backup “qızıl qayda”dır:

```
cp nginx.conf nginx.conf.bak_$(date +%F_%H%M)
```

Əgər date formatı çətin gəlicə, ən azından manual .bak yaratmaq belə faydalıdır.

6.8 recursive copy

Böyük qovluqları kopyalayan zaman:

```
cp -r source_dir/ dest_dir/
```

Risk:

- disk doluluğu
- kopya vaxt aparır

Kopyadan əvvəl ölçünü yoxlamaq faydalıdır:

```
du -sh source_dir
```

6.9 qovluq strukturunu köçürmək

Əgər yalnız struktur lazımdırsa (fayllar yox), find ilə:

```
cd source_dir  
find . -type d -print0 | xargs -0 -I{} mkdir -p "../dest_dir/{"
```

Bu tip komandalar prod-da ehtiyatla işlədilməlidir. Əmin deyilsinizsə, DevOps ilə edin.

6.10 fayl axtarmaq

find

Qovluqda fayl tapmaq:

```
find /opt/wss -name "docker-compose.yml"  
find . -type f -name "*.json"
```

locate

locate sürətlidir, amma database yenilənməlidir (hər serverdə olmaya bilər).

6.11 fayl içində mətn axtarmaq

grep

Fayl içində “sətir tap”:

```
grep "proxy_pass" /etc/nginx/nginx.conf
```

grep -r

Qovluq daxilində recursive axtarış:

```
grep -R "server_name" /etc/nginx/
```

Çıxış çoxdursa:

```
grep -R "server_name" /etc/nginx/ | head
```

6.12 fayl ölçüsü və tarixçəsini görmək

- ölçü və tarix: `ls -lah`
- qovluq ölçüsü: `du -sh`

Misal:


```
ls -lah appsettings.json  
stat appsettings.json
```

Növbəti bölmə fayla baxmaq və redaktə (cat/less/head/tail/nano/vim) və config dəyişəndə nəzarət addımlarını detallandırır.

Bölmə 7. Fayla baxmaq və redaktə etmək

Bu bölmədə məqsəd: serverdə config və log fayllarını “minimum risklə” oxumaq və lazım gələrsə redaktə etmək.

7.1 `cat`

Kiçik fayllar üçün:

```
cat appsettings.json
```

Risk:

- böyük log faylını `cat` etmək terminalı “daşır”. Böyük faylda `less` istifadə edin.

7.2 less

Böyük fayllar üçün ən rahat:

```
less /var/log/nginx/error.log
```

İdarəetmə:

- q → ÇIXIŞ
- /text → axtar
- n → növbəti uyğunluq

7.3 more

less-in sadə alternatividir. Adətən less daha yaxşıdır.

7.4 head

Faylın başlanğıcı:

```
head -n 50 file.log
```

7.5 tail

Faylın sonu:

```
tail -n 200 file.log
```

7.6 `tail -f` ilə log izləmək

Canlı log izləmək:

```
tail -f /var/log/nginx/error.log
```

Qısa yol ilə dayandırmaq: `Ctrl+C`.

Triage zamanı tövsiyə:

- problem replika edilən anda `tail -f` açıq saxlayın
- timestamp-ə diqqət edin

7.7 `nano` ilə redaktə

Sadə editor:

```
nano appsettings.json
```

Əsas düymələr:

- `Ctrl+O` → yaz (save)
- `Ctrl+X` → çıx

Prod-da:

- redaktə etməzdən əvvəl backup yaradın (Bölmə 6.7)

7.8 `vim` ilə baza səviyyəsində iş

Bəzi serverlərdə `nano` olmaya bilər, amma `vim` olar.

Əsas:

- aç: `vim file`
- edit rejimi: `i`
- çıx/saxla: `Esc` sonra `:wq`
- çıx (saxlamadan): `Esc` sonra `:q!`

7.9 config faylını dəyişəndə nəzarət addımları

Config dəyişmə “dəyişiklik”dir. Minimal nəzarət ardıcılığı:

1. Dəyişiklikdən əvvəl backup:

```
cp config.json config.json.bak
```

1. Dəyişiklikləri kiçik saxla (bir dəfəlik böyük refactor etmə).
2. Əgər mümkündürsə config validation et:

- nginx üçün: `nginx -t`
- docker-compose üçün: `docker compose config`

1. Dəyişiklikdən sonra xidmətin davranışını yoxla:

- servis status
- loglarda yeni error varmı
- health endpoint

7.10 dəyişiklikdən əvvəl backup almaq

Backup strategiyası sadə ola bilər:

- eyni qovluqda .bak fayl
- tarixli .bak_YYYY-MM-DD_HHMM

Vacib olan:

- backup-u tapmaq asan olsun
 - rollback zamanı “hansı versiya?” sualına cavab olsun
-

Növbəti bölmə icazələr və istifadəçilər (chmod/chown/sudo/root) mövzusunu daha sistemli verir və “permission denied” xətasını analiz etməyi öyrədir.

Bölmə 8. İcazələr və istifadəçilər

“Permission denied” Linux-da ən çox rast gəlinən problemlərdən biridir. Bu bölmə icazə modelini sadə dildə izah edir.

8.1 user, group, owner anlayışı

Hər faylın:

- owner-i (istifadəçi)
- group-u
- permission set-i var

Yoxlama:

```
ls -la
```

8.2 `chmod`

Permission dəyişmək.

Rəqəmli format:

- $r = 4, w = 2, x = 1$
- $7 = rwx, 6 = rw-, 5 = r-x, 4 = r--$

Tipik:

```
chmod 644 appsettings.json
```

```
chmod 755 /opt/wss/docs
```

8.3 `chown`

Owner/group dəyişmək:

```
sudo chown root:root /etc/nginx/nginx.conf
```

8.4 executable permission

Fayl “işlədilə bilən” olsun deyə x icazəsi lazımdır.

```
chmod +x script.sh
```

```
./script.sh
```

8.5 777 niyə təhlükəlidir

777 deməkdir:

- hamı oxuya, yaza, icra edə bilər

Bu:

- təhlükəsizlik riskidir
- audit zamanı problem yaradır
- bəzən sistem davranışını dəyişir

8.6 755, 644 tipik halları

- qovluq: 755 (owner tam, digərləri oxu+icra)
- config faylı: 644 (owner oxu+yaz, digərləri oxu)

8.7 permission denied xətasının analizi

Ən faydalı 3 sual:

1. Hansı user-lə işləyirəm?

```
whoami  
id
```

1. Faylın owner/permission-ı nədir?

```
ls -la /path/to/file
```

1. Proses hansı user-lə işləyir?

- docker konteynerində proses user-i fərqli ola bilər
- system service-də `systemctl status` məlumat verə bilər

Prod-da ümumi yanaşma:

- icazəni “hamıya açmaq” yox, düzgün user/group-a vermək

8.8 `sudo` **Və** `sudo su`

- `sudo <command>` — tək komandanı admin kimi icra edir
- `sudo su` — root shell açır

Prod intizamı:

- mümkün qədər `sudo <command>` istifadə edin
- root shell-də yanlış əməl riski daha yüksəkdir

8.9 root session ilə işləmə intizamı

- root sessiya açanda məqsədi konkret olsun
- iş bitən kimi çıxın: `exit`
- paralel bir neçə root sessiya açmayın

Növbəti bölmə monitoring: CPU/RAM/disk, proseslər, kill və sistemin “niyə yavaşladığını” ilkin səviyyədə anlama.

Bölmə 9. Sistem resursları və monitoring

Incident zamanı ilk suallardan biri: “problem tətbiqdədir, yoxsa server resurslarında?” Bu bölmə BA üçün resurs yoxlamasını sadə edir.

9.1 top

CPU/RAM istifadə edən prosesləri canlı göstərir:

[top](#)

Çıxış: q.

9.2 htop

top-un daha rahat variantı (hər serverdə olmaya bilər):

htop

9.3 free -m

RAM vəziyyəti:

free -m

Nəyə baxmalı:

- available memory çox azdır?
- swap istifadə olunur?

9.4 df -h

Disk doluluğu:

df -h

Disk 90–95%+ doludursa:

- DB yavaşlaya bilər
- container logları şişə bilər
- servis yazma əməliyyatlarında qıra bilər

9.5 du -sh

Qovluq ölçüsü:

du -sh /var/log

Problemi tapmaq üçün:

```
du -sh /var/log/* | sort -h | tail
```

9.6 uptime

Serverin neçə gündür işlədiyini və load average-i göstərir:

```
uptime
```

Load average yüksəkdirsə, CPU və ya I/O bottleneck ola bilər.

9.7 ps aux

Proses siyahısı:

```
ps aux | head
```

```
ps aux | grep nginx
```

9.8 kill VƏ kill -9

- kill <pid> — prosesi “yumşaq” dayandırmağa çalışır
- kill -9 <pid> — prosesi zorla öldürür

Prod üçün xəbərdarlıq:

- kill -9 data korlanmasına səbəb ola bilər.
- BA təkbaşına process kill etməməlidir (prosedurla).

9.9 CPU, RAM, disk bottleneck necə anlaşılır

Sadə triage:

- CPU: top/htop-da 100% yaxın, load yüksək
- RAM: free -m-də available çox az, swap artır
- Disk:
 - df -h doludur
 - iowait top-da yüksəkdir

Simptomlar:

- UI yavaş
- timeouts
- DB query-lər uzun çəkir

9.10 disk dolması və sistemə təsiri

Disk dolanda:

- log yazmaq mümkün olmur
- DB transaction-lar fail ola bilər
- docker overlay storage problem yarada bilər

BA üçün ən təhlükəsiz addımlar:

- df -h ilə təsdiqlə
- hansı qovluq şişib du ilə tap
- silmə əməliyyatına tələsmə (log silmək audit riskidir)

- DevOps ilə razılaşdır
-

Növbəti bölmə şəbəkə əsasları: IP/port/hostname, ping/curl, port dinləmə (ss -tuln), traceroute və local/remote bağlantı məntiqi.

Bölmə 10. Linux şəbəkə əsasları

Şəbəkə problemləri çox vaxt “servis down” kimi görünür, amma səbəb DNS/firewall/routing olur. Bu bölmə BA üçün minimal şəbəkə triage verir.

10.1 IP, port, hostname

- IP: serverin şəbəkə ünvanı
- Port: eyni IP-də fərqli servisləri ayırmaq üçün “qapı” (məs: 22 SSH, 443 HTTPS)
- Hostname: DNS adı (məs: docs.company.az)

10.2 private subnet nədir

Private subnet adətən internetə açıq olmayan daxili şəbəkədir:

- 10.0.0.0/8
- 172.16.0.0/12
- 192.168.0.0/16

Bu şəbəkələrə çatmaq üçün:

- VPN
- jump/bastion
- daxili routing

lazım ola bilər.

10.3 eyni subnetdə iki server necə danışır

Eyni subnetdə (məs: 10.0.10.x) serverlər adətən birbaşa danışa bilir (firewall bloklamırsa).

Fərqli subnetdə isə routing qaydası lazımdır.

10.4 ping

Network “ümumi əlçatımlılıq” yoxlaması:

```
ping -c 4 10.0.10.25
```

Qeyd:

- bəzən ping firewall tərəfindən bloklanır, ping işləmirsə bu mütləq “server down” demək deyil.

10.5 curl

HTTP/HTTPS endpoint yoxlaması:

```
curl -I https://docs.company.az
```

```
curl -v http://127.0.0.1:5000/health
```

-I yalnız header-ləri gətirir.

10.6 ss -tuln

Serverdə hansı portlar “listen” edir:

```
ss -tuln
```

Filtrləmək:

```
ss -tuln | grep 443
```

BA üçün fayda:

- Nginx 443-də dinləyirmi?
- App portu açıqdırmı?

10.7 traceroute və route anlayışı

traceroute (hər serverdə olmaya bilər) paketlərin hansı hop-lardan keçdiyini göstərir.

```
traceroute 10.0.10.25
```

Routing problemi şübhəsi varsa, DevOps/Network komandası ilə işləmək lazımdır.

10.8 local və remote bağlantı məntiqi

Eyni hostda iki nöqtə:

- local: 127.0.0.1 yalnız həmin serverin özündən görünür
- remote: serverin real IP-si ilə digər maşınlar da qoşula bilər (firewall icazə verirsə)

Misal:

- app 127.0.0.1:5000-də dinləyirsə, Nginx eyni hostdadırsa proxy edə bilər.
- amma başqa serverdən birbaşa :5000-ə çıxmaq mümkün olmaya bilər.

Növbəti bölmə firewall və UFW: status yoxlama, port açmaq, source IP üzrə icazə, qayda silmək və təhlükəsiz qayda yazmaq prinsipləri.

Bölmə 11. Firewall və UFW

Firewall şəbəkə trafikinə “icazə ver / blokla” qaydaları tətbiq edir. Docs5 kimi sistemlərdə port problemi çox vaxt firewall ilə bağlı olur.

Vacib: Prod-da firewall qaydası dəyişmək yüksək riskli əməliyyatdır. BA bunu adətən tək etmir; burada məqsəd oxuma/yoxlama və problemi düzgün anlamaqdır.

11.1 firewall nədir

Firewall:

- hansı portdan giriş/çıxış icazəlidir
- hansı IP-lər qoşula bilər
- default olaraq bloklayır və ya icazə verir (policy)

11.2 UFW nədir

UFW (Uncomplicated Firewall) Ubuntu/Debian-da firewall idarəsi üçün sadə interfeysdir.

11.3 `ufw status`

Status və qaydalar:

```
sudo ufw status
```

```
sudo ufw status verbose
```

11.4 `ufw enable / disable`

```
sudo ufw enable
```

```
sudo ufw disable
```

Prod qeydi:

- `disable` etmək təhlükəlidir (bütün açıq portlar exposed ola bilər).

11.5 port açmaq

Məsələn, 443 (HTTPS) açmaq:

```
sudo ufw allow 443/tcp
```

SSH üçün diqqət:

- SSH portunu bağlasanız, serverə girişi itirə bilərsiniz.

11.6 source IP üzrə icazə vermək

Yalnız müəyyən IP-dən icazə:

```
sudo ufw allow from 10.0.10.10 to any port 5432 proto tcp
```

Bu yanaşma daha təhlükəsizdir.

11.7 qaydanı silmək

Ən rahat:

```
sudo ufw status numbered
```

Sonra nömrə ilə silmək:

```
sudo ufw delete 3
```

11.8 default policy

Default policy:

```
sudo ufw default deny incoming
```

```
sudo ufw default allow outgoing
```

Bu, tipik “təhlükəsiz default”dur.

11.9 deny vs allow

- allow — icazə verir
 - deny — bloklayır
-

Bəzən qaydaların sırası və daha spesifik qayda davranışı dəyişir; buna görə “niyə yenə bağlanmır?” halları ola bilər.

11.10 firewall problemi necə anlaşılır

Simptomlar:

- eyni serverdə servis işləyir, amma başqa serverdən qoşulmaq olmur
- connection timed out

Yoxlama yanaşması:

1. Servis portda dinləyirmi?

```
ss -tuln | grep 5432
```

1. UFW status nədir?

```
sudo ufw status verbose
```

1. Local yoxlama:

```
curl -v http://127.0.0.1:5000
```

Local işləyib remote işləmir → firewall/routing ehtimalı artır.

11.11 servis açıqdır, amma port niyə görünmür

Səbəblər:

- servis yalnız 127.0.0.1-ə bind olub
- docker port publish edilməyib

- firewall inbound bloklayır
- security group / network ACL (cloud mühitlərdə)

11.12 təhlükəsiz qayda yazma prinsipləri

- mümkün qədər dar icazə (yalnız lazım olan port, yalnız lazım olan source IP)
- dəyişiklikdən əvvəl təsir analiz
- dəyişiklikdən sonra test:
 - port açıldı?
 - kənardan qoşulma oldu?
- rollback planı (qaydanı silmək)

Növbəti bölmələr arxivləmə, fayl transferi, wget/curl, Docker/Compose və container troubleshooting mövzularına keçəcək.

Bölmə 12. Arxivləmə və sıxma əməliyyatları

Arxivləmə (tar/zip) ən çox bu işlər üçün lazım olur:

- backup hazırlamaq
- böyük qovluğu köçürmədən əvvəl “tək fayla” yığmaq
- Windows-a ötürmək üçün paketləmək

12.1 tar nədir

tar qovluq/faylları “arxiv” adı ilə bir fayla yığır. tar özü sıxma etmir — sadəcə paketləyir.

12.2 tar.gz nədir

tar.gz:

- əvvəl tar ilə paket
- sonra gzip ilə sıxma

Nəticə: daha kiçik fayl.

12.3 zip və tar fərqi

- Linux-da tar.gz daha çox istifadə olunur
- Windows-da zip daha rahat açıla bilər

Praktik:

- server-to-server transfer üçün tar.gz çox uyğundur

12.4 qovluğu tar etmək

```
tar -cvf logs.tar /var/log/nginx
```

- c create
- v verbose
- f file

12.5 tar.gz ilə compress etmək

```
tar -czvf logs.tar.gz /var/log/nginx
```

- z gzip

12.6 archive açmaq

```
tar -xvzf logs.tar.gz
```

Əgər başqa qovluğa açmaq istəsəniz:

```
mkdir -p /tmp/extract
```

```
tar -xvzf logs.tar.gz -C /tmp/extract
```

12.7 list məzmunu görmək

Açmadan içini görmək:

```
tar -tzf logs.tar.gz | head
```

12.8 backup üçün arxivləmə ssenariləri

Təhlükəsiz yanaşma:

- backup-u tarixlə adlandırın
- haradan alındığını adında qeyd edin

```
tar -czvf docs5_nginx_conf_$(date +%F_%H%M).tar.gz /etc/nginx
```

12.9 böyük qovluqları köçürmədən əvvəl yığmaq

Böyük qovluğu bir fayla yığanda:

- transfer daha stabil olur
- integrity check daha rahat olur (checksum)

Checksum nümunəsi:

```
sha256sum logs.tar.gz > logs.tar.gz.sha256
```

12.10 Windows-a ötürmək üçün arxiv hazırlamaq

Windows tərəfi üçün zip lazım ola bilər:

```
zip -r logs.zip /var/log/nginx
```

Əgər zip yoxdursa, qurmaq lazım ola bilər.

Növbəti bölmələr fayl transferinə keçir: scp, rsync, Windows↔Linux sse-nariləri.

Bölmə 13. Linux-dan Linux-a fayl ötürmə

Serverlər arasındakı fayl ötürmənin ən tipik yolu SSH üstündən işləyən `scp`-dir. Daha böyük və davam edən transferlərdə `rsync` daha yaxşıdır.

13.1 `scp` nədir

`scp` (secure copy) SSH üstündən fayl kopyalayır.

13.2 bir serverdən digərinə `scp` ilə fayl atmaq

Lokaldan (sizdəki maşından) serverə:

```
scp file.txt user@10.0.10.25:/tmp/
```

Serverdən lokal maşına:

```
scp user@10.0.10.25:/var/log/nginx/error.log ./
```

Port fərqlidirsə:

```
scp -P 2222 file.txt user@10.0.10.25:/tmp/
```

13.3 qovluq ötürmək

Recursive:

```
scp -r /tmp/debug user@10.0.10.25:/tmp/
```

13.4 rsync giriş səviyyəsi

rsync:

- kəsilib davam edə bilər
- yalnız dəyişən hissəni ötürür
- daha stabildir

Sadə nümunə:

```
rsync -avz /tmp/debug/ user@10.0.10.25:/tmp/debug/
```

SSH port fərqlidirsə:

```
rsync -avz -e "ssh -p 2222" /tmp/debug/ user@10.0.10.25:/tmp/debug/
```

13.5 eyni subnetdə sürətli müvəqqəti transfer yanaşması

Eyni subnetdə böyük fayl ötürmədə:

- disk doluluğunu yoxlayın (`df -h`)
- əvvəl arxiv edin (`tar .gz`)
- checksum ilə təsdiqləyin

13.6 SSH üzərindən transfer riskləri

- səhv serverə göndərmək (mühit qarışır: test vs prod)
- overwrite risk
- secret-lı faylların (vault settings) yanlış paylaşımı

Minimal intizam:

- hədəf IP/hostname-i iki dəfə yoxlayın
- `/tmp` kimi müvəqqəti qovluqdan istifadə edin
- prod secret fayllarını mail/chat ilə paylaşmayın

Növbəti bölmə Windows↔Linux transfer ssenarilərini izah edir.

Bölmə 14. Linux-dan Windows-a və Windows-dan Linux-a fayl ötürmə

Bu bölmə BA üçün ən çox lazım olan “logu serverdən götürüb Windows-da paylaşmaq” ssenarisini sadələşdirir.

14.1 Windows-da `scp` istifadəsi

Müasir Windows 10/11-də OpenSSH client çox vaxt hazır gəlir.

PowerShell/CMD-də:

```
scp user@10.0.10.25:/var/log/nginx/error.log .
```


14.2 PowerShell üzərindən scp

Port fərqlidirsə:

```
scp -P 2222 user@10.0.10.25:/var/log/nginx/error.log .
```

Key ilə:

```
scp -i C:\Users\you\.ssh\id_rsa user@10.0.10.25:/tmp/report.txt .
```

14.3 Linux serverdən Windows-a fayl endirmək

Windows tərəfdə scp ilə endirmək adətən daha rahatdır.

Alternativ:

- WinSCP (Bölmə 4)

14.4 Windows-dan Linux serverə fayl atmaq

```
scp .\appsettings.json user@10.0.10.25:/tmp/
```

Prod-da risk:

- config faylını yanlış yerə atmaq
- overwrite etmək

14.5 path format fərqləri

- Windows: C:\Users\you\file.txt

- Linux: `/home/you/file.txt`

PowerShell-də qaçış simvollarına diqqət:

- `\` iki backslash ola bilər

14.6 tipik `scp` xətaları

- `Permission denied` → user/key problem
- `Connection timed out` → VPN/firewall/jump tələb olunur
- `No such file or directory` → path səhvdir (Linux path-ları case-sensitive-dir)

14.7 böyük fayllar üçün tövsiyələr

- əvvəl arxiv edin (`tar.gz` və ya `zip`)
- checksum ilə təsdiqləyin
- mümkün olduqda `rsync` istifadə edin (Linux tərəfdə)

Növbəti bölmə “müvəqqəti fayl paylaşımı” üçün Python HTTP server + `wget` yanaşmasını izah edir.

Bölmə 15. Müvəqqəti fayl paylaşımı: Python HTTP server + wget

Bəzən scp ilə problem olur (jump, key, policy) və sizə qısa müddətlik “serverdə qovluğu HTTP ilə paylaşım, digər serverdən çekim” yanaşması lazım olur.

Təhlükəsizlik xəbərdarlığı: HTTP server açmaq “paylaşım” deməkdir. Prod-da bunu yalnız daxili şəbəkədə, minimal vaxt və firewall nəzarəti ilə edin.

15.1 Bu yanaşma nə vaxt lazımdır

- eyni subnetdə iki server arasında sürətli transfer
- `scp` policy ilə bloklanıb
- müvəqqəti log/artefakt paylaşmaq lazımdır

15.2 Python ilə qovluğu host etmək

Qovluğa keçin:

```
cd /tmp/debug
```

Python 3 ilə server açın:

```
python3 -m http.server 8080
```

Bu, həmin qovluğu `http://<server-ip>:8080/` ilə paylaşır.

15.3 hansı portda publish etmək

- 8080 çox vaxt rahatdır
- amma təşkilatın policy-si və firewall qaydaları nəzərə alınmalıdır

15.4 digər serverdən `wget` ilə çəkmək

Digər serverdə:

```
wget http://10.0.10.25:8080/error.log
```

15.5 lokal IP ilə fayl ötürmə ssenarisi

Ən təhlükəsiz:

- yalnız private IP
- yalnız daxili subnet
- mümkün olduqda source IP məhdudiyyəti

15.6 firewall və port yoxlaması

Host tərəfdə port dinləyirmi?

```
ss -tuln | grep 8080
```

Firewall açıqdır?

```
sudo ufw status
```

15.7 directory listing riskləri

Python http server default olaraq directory listing verir. Bu riskdir.

Minimal tədbirlər:

- /tmp kimi müvəqqəti qovluq istifadə edin
- yalnız lazım olan faylları qoyun
- iş bitən kimi serveri bağlayın

15.8 iş bitəndən sonra host-un bağlanması

Terminalda Ctrl+C.

15.9 `wget` ilə tək fayl çəkmək

```
wget -O nginx_error.log http://10.0.10.25:8080/error.log
```

15.10 `wget` ilə recursive çəkmə əsasları

Recursive risklidir (çox fayl çəkə bilər):

```
wget -r -np -nH --cut-dirs=0 http://10.0.10.25:8080/
```

Prod-da recursive istifadə etməzdən əvvəl məzmunu dəqiqləşdirin.

Növbəti bölmə `wget` və `curl` ilə endpoint yoxlama və fayl endirmə mövzularını genişləndirir.

Bölmə 16. `wget` və `curl` ilə fayl və endpoint işi

`wget` və `curl` BA üçün “API işləyir?” və “bu URL-dən fayl endirə bilərəm?” suallarına cavab verən əsas alətlərdir.

16.1 `wget` nədir

- əsasən fayl endirmək üçün
- HTTP/HTTPS üstündən işləyir

16.2 `curl` və `wget` fərqi

- `curl` daha çox “request yoxlama” (header, status code, auth)
- `wget` daha çox “download”

Amma ikisi də çox şey edə bilir.

16.3 URL-dən fayl endirmək

`wget https://example.com/file.tar.gz`

16.4 output file təyin etmək

`wget -O backup.tar.gz https://example.com/download?id=123`

16.5 self-signed / internal certificate halları

Internal TLS-də bəzən sertifikat trust olunmur.

curl ilə:

`curl -vk https://internal.service/health`

- -k certificate verify-ni keçər (yalnız test/diagnostic üçün)

Prod təhlükəsizliyi: -k “normal işləmə” üçün yox, yalnız diaqnoz üçündür.

16.6 auth tələb edən endpoint-lər

Header ilə token:

`curl -H "Authorization: Bearer <TOKEN>" https://docs.company.az/api/me`

X-API-KEY nümunəsi:


```
curl -H "X-API-KEY: <KEY>" https://docs.company.az/api/ping
```

Secret-ları komanda sətrində yazmaq risklidir (history). Mümkün olduqda dəyişənlə:

```
export API_KEY="..."  
curl -H "X-API-KEY: $API_KEY" https://...
```

16.7 connection troubleshoot

Tipik yoxlamalar:

- DNS?

```
getent hosts docs.company.az
```

- Port açıqdır?

```
ss -tuln | grep 443
```

- HTTP status nədir?

```
curl -I https://docs.company.az
```

- Proxy arxasında upstream problemi varmı?

```
curl -v https://docs.company.az
```

Növbəti bölmələr Docker-a keçir: image/container/volume/network anlayışları, docker ps, loglar və Compose əməliyyatları.

Bölmə 17. Docker əsasları

Docker5-in bir çox yerləşdirməsində servis(lər) Docker konteynerlərində işləyir. BA üçün məqsəd:

- konteynerlərin vəziyyətinə baxmaq
- log oxumaq
- minimal restart/stop/start fərqlərini başa düşmək

17.1 Docker nədir

Docker tətbiqləri “container” içində işlətməyə imkan verir.

- container → izolyasiya olunmuş proses mühiti
- image → container-in “şablonu”

17.2 image, container, volume, network anlayışı

- Image: hazır paket (OS layer + app)
- Container: image-dən işləyən instans
- Volume: persistent data (DB data, upload file, və s.)
- Network: container-lər arası əlaqə

17.3 container və VM fərqi

Sadə izah:

- VM: ayrıca virtual OS
- Container: eyni host kernel üstündə izolyasiya

Praktik nəticə:

- container daha yüngül və sürətlidir
- amma host resurslarına bağlıdır

17.4 Docs5 niyə docker üzərində qurulur

- deploy daha standart olur
- servis-ləri ayrılıqda idarə etmək asanlaşır
- upgrade/migration util-lərini ayrıca konteyner kimi işə salmaq mümkündür

Bu repodakı Sources/utils/*/docker-compose.yml fayllarında “dotnet util” konteynerinin bir dəfə işləyib çıxması nümunələri var.

17.5 docker version və docker info yoxlaması

`docker version`

`docker info`

BA üçün:

- docker işləyirmi?
- host resurs limitləri varmı?

17.6 docker ps

İşləyən konteynerlər:

`docker ps`

17.7 docker ps -a

Hamısı (dayananlar da):

`docker ps -a`

Triage:

- konteyner tez-tez restart edirsə → problem var

17.8 docker images

Image siyahısı:

```
docker images
```

17.9 container lifecycle

Əsas əməllər:

```
docker start <container>
```

```
docker stop <container>
```

```
docker restart <container>
```

```
docker rm <container>
```

Risk qeydi:

- rm konteyneri silir; volume silinmirsə data qalır, amma yenə də ehtiyatlı olun.

17.10 docker inspect

Konteynerin detalları (network, mount, env):

```
docker inspect <container>
```

Buradan:

- hansı volume mount edilib
- hansı IP/port-lar var

tapmaq olar.

17.11 docker stats

Konteyner resurs istifadəsi:

`docker stats`

CPU/RAM limit problemi şübhəsi varsa faydalıdır.

Növbəti bölmə Docker Compose: servis-ləri birlikdə qaldırmaq/endirmək, log/ps/config, rebuild halları və tipik problemlər.

Bölmə 18. Docker Compose ilə işləmək

Docker Compose bir neçə konteyneri “bir paket” kimi idarə etmək üçündür. Docs5 kimi sistemlərdə Nginx, app, DB, Keycloak, util-lər çox vaxt compose ilə ayağa qaldırılır.

18.1 `docker compose` nədir

Compose sizə `docker-compose.yml` faylında servis-ləri təsvir edib, sonra bir komanda ilə hamısını işə salmağa imkan verir.

18.2 `docker-compose` və `docker compose` fərqi

- `docker compose` → Docker CLI-nin yeni integrasiyası
- `docker-compose` → köhnə standalone binary

Bu kitabçada əsasən docker compose formasını istifadə edirik.

18.3 compose file strukturu

Tipik:

- services: konteynerlər
- volumes: persistent data
- networks: şəbəkələr

18.4 services

Hər servis:

- image və ya build
- environment
- volumes
- ports
- depends_on

18.5 ports

Port mapping nümunəsi:

- 443:443 → host 443 → container 443

18.6 environment

Config çox vaxt environment variable-lərlə verilir. Secret-ları .env və ya Vault ilə idarə etmək daha doğrudur.

18.7 volumes

Volume:

- DB data itmesin deyə
- upload/storage data qalıcı olsun deyə

18.8 networks

Servislər eyni network-dədirsə, bir-birini servis adı ilə görə bilər.

18.9 docker compose up -d

Servisləri background-da qaldırmaq:

```
docker compose up -d
```

18.10 docker compose down

Servisləri endirmək:

```
docker compose down
```

Qeyd: down şəbəkəni də silə bilər. Volume-ların silinməsi ayrıca flag-larla olur; yenə də ehtiyatlı olun.

18.11 docker compose stop

Servisləri dayandırır, amma “təsvir” saxlanır:

```
docker compose stop
```

18.12 docker compose restart

```
docker compose restart
```

Restart “müvəqqəti fix” kimi cazibədardır, amma root cause tapmadan prod-da ardıcıl restart etmək düzgün deyil.

18.13 docker compose logs -f

Logları izləmək:

```
docker compose logs -f
```

Tək servis:

```
docker compose logs -f nginx
```

18.14 docker compose ps

Compose servis status:

```
docker compose ps
```

18.15 config dəyişikliyindən sonra nə etmək

Dəyişiklik növünə görə:

- yalnız mount olunan fayl dəyişibsə → bəzən restart yetər
- image/build dəyişibsə → up -d (bəzən --build)

Təhlükəsiz ardıcılıq:

1. backup
2. config validate (mümkündürsə)
3. kontrollu restart
4. log + health check

18.16 rebuild halları

```
docker compose up -d --build
```

18.17 orphan container və digər tipik problemlər

Orphan container:

- compose faylından servis silinib, amma köhnə konteyner qalıb

Həll:

```
docker compose up -d --remove-orphans
```

Repo-dakı util compose nümunəsi

Bu repoda util-lər üçün compose nümunələri var (sadə “bir dəfə işləyib çıxan” konteyner):

```
services:
  wssutil:
    image: images.wss-consulting.ru/dot/docs/prod:latest
    restart: "no"
    volumes:
      - "./appsettings.json:/app/appsettings.json:ro"
      - "../..../vault.settings.json:/app/vault.settings.json:ro"
    entrypoint: ["dotnet", "util.dll"]
```

BA üçün nəticə:

- bu util-lər up edilib log oxunur
- sonra konteyner exit code-a baxılır

Növbəti bölmə container daxilində işləmək: `docker exec`, `bash` vs `sh`, container içindən `config/log` tapmaq və `docker cp` ilə fayl köçürmək.

Bölmə 19. Container daxilində işləmək

Bəzən log və config hostda deyil, konteynerin içində olur. Bu bölmədə konteynerə daxil olub read-only yoxlama etməyi öyrənirsiniz.

19.1 `docker exec -it`

Konteynerə shell açmaq:

```
docker exec -it <container_name> bash
```

Əgər bash yoxdursa:

```
docker exec -it <container_name> sh
```

19.2 bash və sh fərqi

- bash daha funksional shell
- sh daha sadə, minimal image-lərdə olur

19.3 container daxilində fayl tapmaq

Container içində:

```
find / -name "appsettings.json" 2>/dev/null | head
```

19.4 container daxilində config oxumaq

```
cat /app/appsettings.json
```

Əgər config volume ilə mount olunubsa, hostda da ola bilər.

19.5 container daxilində log faylı axtarmaq

```
find /var/log -type f 2>/dev/null | head
```

Amma çox servis logu stdout-a yazır — o zaman docker logs daha doğru yerdir.

19.6 container daxilində command işlətmək

Shell açmadan da icra etmək olar:

```
docker exec <container_name> ls -la /app
```

19.7 container içərisinə fayl yerləşdirmək (docker cp)

Hostdan konteynerə:

```
docker cp ./appsettings.json <container_name>:/app/appsettings.json
```

Prod-da bu adətən tövsiyə olunmur, çünki container içində dəyişiklik “persistent” olmaya bilər. Daha doğru yol: volume mount + controlled deploy.

19.8 container-dən host-a fayl çıxarmaq

```
docker cp <container_name>:/app/appsettings.json ./appsettings.json
```

Bu, audit/troubleshooting üçün faydalıdır.

19.9 container daxilində müvəqqəti debug

Debug edərkən:

- env ilə environment variable-ləri yoxlamaq
- cat ilə config oxumaq

Amma:

- prod-da əlavə paket install etmək risklidir
- mümkün olduqda host tərəfdə yoxlama edin

19.10 dəyişikliklərin persistent olub-olmaması

Əsas prinsip:

- container içində etdiyiniz dəyişiklik image-də deyil, konteynerin “üst layer”-ində qalır
- konteyner silinib yenidən yaradılsa, dəyişiklik itə bilər

Ona görə prod-da:

- dəyişikliklər compose/deploy yolu ilə edilməlidir
-

Növbəti bölmə container logları və troubleshooting: `docker logs`, restart loop, exit code, “Restarting (139)” kimi halların mənası və root cause tapmaq.

Bölmə 20. Container logları və troubleshooting

Konteyner “ayaqdadır” demək “servis işləyir” demək deyil. Burada məqsəd logdan root cause tapmaq üçün sistematik yanaşmadır.

20.1 `docker logs`

Konteynerin stdout/stderr logları:

```
docker logs <container_name>
```

20.2 `docker logs -f`

Canlı izləmə:

```
docker logs -f <container_name>
```

20.3 son sətirləri görmək

```
docker logs --tail 200 <container_name>
```

20.4 restart loop-ları anlamaq

Əlamətlər:

- `docker ps-də` Restarting status
- uptime çox kiçik, restart count artır

Nə etmək:

- `docker logs --tail 200` ilə son error-u tap
- config/secret çatışmır?
- port conflict?

20.5 exit code anlayışı

Konteyner çıxanda exit code verir.

Baxmaq:

```
docker inspect <container_name> --format '{{.State.ExitCode}}'
```

Tipik:

- 0 → uğurlu çıxış
- 1 → ümumi error

20.6 “Restarting (139)” kimi halların mənası

139 çox vaxt segmentation fault (crash) kimi anlaşılr.

Tipik səbəblər:

- native library problemi
- memory corruption
- image uyğunluğu

BA üçün addımlar:

- logları saxla
- son deploy dəyişiklikləri soruş
- DevOps/Dev-ə eskalasiya

20.7 logdan root cause tapmaq

Yaxşı yanaşma:

1. Ən son error sətirini tap
2. Error-un kontekstini (10–20 sətir əvvəl/sonra) çıxar
3. Timestamp-ə bax
4. “config path”, “connection string”, “permission denied” kimi açar sözlər axtar

20.8 app log vs proxy log vs system log

- Nginx logları: `/var/log/nginx/access.log`, `/var/log/nginx/error.log`

- App logları: container stdout (`docker logs`) və ya app içi log
- System log: `journalctl` (systemd olan serverlərdə)

Simptom → log:

- 502 → əvvəl Nginx error.log
- DB error → app log
- disk dolu → system log + `df -h`

20.9 container daxilində müvəqqəti debug

Əgər container dərhal çıxırsa:

- `entrypoint/command` səhv ola bilər
- `config mount` olunmur

Compose ilə debug üçün bəzən:

- `docker compose run --entrypoint sh <service>` kimi yanaşmalar istifadə olunur (prosedurla).

20.10 dəyişikliklərin persistent olub-olmaması

Troubleshooting zamanı:

- “konteyner içində düzəltdim” həlli prod üçün risklidir
 - düzəliş `deploy/compose` səviyyəsində edilməlidir
-
-

Növbəti bölmələr Docs5 qovluq/servis strukturu, Nginx reverse proxy, PostgreSQL qoşulma və DB əməliyyatları, Postman testləri və auth (Vault/Keycloak) mövzularına keçəcək.

Bölmə 21. Docs5 qovluq və servis strukturu

Bu bölmə Docs5-in serverdə “harada yaşadığını” anlamaq üçündür. Incident zamanı ən çox vaxt itirən şey: doğru qovluğu, doğru compose faylını, doğru log yerini tapa bilməməkdir.

Qeyd: Real yerləşim təşkilata görə dəyişə bilər. Burada ən çox rast gəlinən “/opt/wss/...” modelini izah edirik.

21.1 /opt/wss/docs

Bu qovluq çox vaxt Docs5 tətbiqinin (və ya onun deploy artefaktlarının) yerləşdiyi əsas qovluq olur.

Burada ola biləcək şeylər:

- `docker-compose.yml` və ya bir neçə `compose` faylı
- `.env` və ya `environment` faylları
- `appsettings.json` və əlavə `config` fayllar
- `upgrade/migration` util-lərinin qovluqları

BA üçün praktik:

- Əgər “servis qaldırmaq” lazımdırsa, əvvəlcə `compose` faylının harada olduğunu tapın.
- Əgər “son dəyişiklik nə idi?” sualı varsa, bu qovluqda `timestamp`-lərə (`ls -la`) baxmaq faydalıdır.

21.2 `/opt/wss/storage`

Storage servisi və ya `storage data` çox vaxt ayrıca yerdə saxlanılır.

Burada ola bilər:

- `storage` servis `config`-i
- `storage volume mount`-ları
- `blob` faylları (siyasətə görə)

Təhlükəsizlik:

- `Storage` qovluğunda “təsadüfi silmə” böyük `data` itkisi yarada bilər.
- BA üçün düzgün addım: ölçü yoxlama (`du -sh`) + eskalasiya.

21.3 /opt/wss/vault

Əgər Vault ayrıca servis kimi deploy olunubsa, onun data/config yerləri burada ola bilər.

BA üçün:

- Vault down olanda bir çox servis “config tapa bilmir” kimi görünə bilər.

21.4 /opt/wss/keycloak

Keycloak yerləşimi:

- Keycloak container + data volume
- realm export/import faylları (bəzən)

21.5 /opt/wss/postgres **və ya system PostgreSQL**

PostgreSQL iki modeldə olur:

1. Docker container içində
2. Host OS-də system service kimi (systemctl ilə)

BA üçün fərq:

- container modeldə: `docker ps`, `docker logs`
- system modeldə: `systemctl status postgresql`, `journalctl -u postgresql`

21.6 nginx config yerləri

Nginx adətən:

- host-da quraşdırılırsa: `/etc/nginx/`
- container-dədirsə: `compose volume`-ları vasitəsilə hostdan mount olunur

Tipik:

- `/etc/nginx/nginx.conf`
- `/etc/nginx/conf.d/*.conf`
- `/etc/nginx/sites-enabled/*` (Ubuntu-da)

21.7 env və `vault.settings` faylları

Docs5-in çox hissəsində “real secret” config-lar:

- `.env` (`compose` üçün)
- `vault.settings.json` (application üçün)

Bu repodakı util-lər bunu göstərir:

- `appsettings.json` içində
`"PrependConfigFiles": ["vault.settings.json"]`
- `compose`-da `../.. /vault.settings.json:/app/vault.settings.json:ro` mount edilir

BA üçün vacib qayda:

- `vault.settings.json` mətnini chat/email ilə paylaşmayın.

- log və screenshot-larda credential görünməsin.

21.8 hansı servis hansı domainə cavab verir

Əsas sual:

- docs.company.az hansı servisin upstream-idir?

Cavab adətən Nginx config-dədir:

- server_name docs.company.az;
- proxy_pass http://...;

Növbəti bölmə Nginx reverse proxy və ən tipik 502 problemlərinin analizinə keçəcək.

Bölmə 22. Nginx və reverse proxy əsasları

Nginx Docs5 kimi sistemlərdə çox vaxt “giriş qapısı”dır. İstifadəçi domənə girir və ilk qarşılayan Nginx olur.

22.1 reverse proxy nədir

Reverse proxy:

- istifadəçi request-lərini qəbul edir
- daxildəki servise yönləndirir
- TLS (HTTPS) terminasiya edə bilər
- cache, gzip, rate limit kimi funksiyalar verə bilər

22.2 upstream nədir

Upstream — Nginx-in yönləndirdiyi daxili hədəf.

Misal:

- 127.0.0.1:5000
- docs-app:5000 (docker network daxilində)

22.3 proxy_pass

Ən tipik direktiv:

```
location / {  
    proxy_pass http://127.0.0.1:5000;  
}
```

Burada:

- Nginx request-i app portuna ötürür.

22.4 80 və 443 məntiqi

- 80 → HTTP
- 443 → HTTPS

Tipik pattern:

- 80-də gələn request 443-ə redirect edilir

22.5 SSL certificate və private key

HTTPS üçün:

- certificate (public)
- private key (gizli)

Private key təhlükəsizlik baxımından çox həssasdır:

- paylaşılır
- repo-ya commit olunmur

22.6 `server_name`

Domen match:

```
server {  
    server_name docs.company.az;  
}
```

Əgər `server_name` uyğun gəlmirsə:

- default server cavab verə bilər
- 404/incorrect site görünə bilər

22.7 nginx config test

Dəyişiklikdən sonra test:

```
sudo nginx -t
```

Uğurlu olsa:

```
sudo systemctl reload nginx
```

reload config-i yenidən oxuyur, connection-ları daha az pozur. restart daha sərtidir.

22.8 common 502 səbəbləri

502 “Bad Gateway” deməkdir: Nginx upstream-dən düzgün cavab ala bilmir.

Tipik səbəblər:

- app servisi down
- upstream port dəyişib
- firewall/selinux bloklayır
- app crash edir (restart loop)

22.9 upstream connection refused

Nginx error log-da tipik görünüş:

- `connect() failed (111: Connection refused) while connecting to upstream`

Bu çox vaxt deməkdir:

- həmin portda heç nə dinləmir (`ss -tuln` ilə yoxlanır)

22.10 nginx log oxuma əsasları

Nginx log yerləri deploy-a görə dəyişir.

Host Nginx:

- `/var/log/nginx/access.log`
- `/var/log/nginx/error.log`

Konteyner Nginx:

- `docker logs nginx`

Triage qaydası:

- 502 → əvvəl error log
- 404/redirect → access log (request path, host header)

Növbəti bölmələr PostgreSQL-ə qoşulma (pgAdmin, SSH tunnel, psql) və təhlükəsiz read-only DB yoxlamalarıdır.

Bölmə 23. PostgreSQL-ə qoşulma: pgAdmin və digər üsullar

Bu bölmədə məqsəd: BA-nın PostgreSQL-ə təhlükəsiz şəkildə qoşulub minimal yoxlamaları edə bilməsidir.

23.1 PostgreSQL connection anlayışı

Bir DB connection üçün əsas parametrlər:

- host
- port (default 5432)
- database name
- username

- password

23.2 host, port, db name, username, password

Nümunə (placeholder):

- host: 10.0.10.30
- port: 5432
- db: DBF_Content
- user: wss_docs_db

Vacib: Password/connection string-ləri açıq mətnlə paylaşmayın.

23.3 pgAdmin nədir

pgAdmin:

- GUI vasitəsilə PostgreSQL idarəetmə aləti
- query yazmaq, table-lara baxmaq, connection test etmək olar

23.4 pgAdmin ilə yeni server connection yaratmaq

1. Register -> Server
2. General tab: Name verin (məs: prod-db (via ssh))
3. Connection tab: host/port/db/user daxil edin

23.5 pgAdmin-də General və Connection hissələri

- General: yalnız ad və qruplama
- Connection: real texniki parametrlər

23.6 SSH tunnel ilə qoşulma məntiqi

DB server çox vaxt birbaşa açıq olmur. Onda:

- lokal kompüterdən jump/bastion-a SSH
- jump serverdən DB-yə daxili şəbəkə ilə çıxış

Bu “tunnel” yaradır: siz lokalda `localhost:xxxx`-ə qoşulursunuz, trafikiniz DB-yə gedir.

23.7 Nə zaman SSH tunnel lazımdır

- DB private subnet-dədir
- firewall yalnız jump serverdən icazə verir

23.8 pgAdmin içində SSH tunneling konfigurasiyası

pgAdmin versiyasına görə menyu fərqli ola bilər, amma məntiq eynidir:

- SSH Host: jump server

- SSH Username: jump user
- SSH Key/Password: authentication
- DB Host: daxildəki DB host (bəzən 127.0.0.1 jump tərəfindən)

23.9 local port forwarding məntiqi

CLI ilə örnək:

```
ssh -L 15432:10.0.10.30:5432 ba@jump.company.az
```

Sonra pgAdmin-də:

- host: 127.0.0.1
- port: 15432

23.10 connection failure halları

- timeout → routing/firewall/VPN
- password authentication failed → credential
- no pg_hba.conf entry → DB access policy

23.11 `psql` ilə alternativ qoşulma

Serverdə `psql` varsa:

```
psql -h 10.0.10.30 -p 5432 -U wss_docs_db -d DBF_Content
```

Password soruşa bilər.

Təhlükəsiz yanaşma:

- prod-da mümkün olduqda read-only user istifadə edin

23.12 DB list və table list yoxlaması

psql içində:

- DB list: \l
- table list: \dt

23.13 readonly işləmək intizamı

BA üçün əsas:

- SELECT + LIMIT ilə yoxla
- UPDATE/DELETE etmə
- şübhəli script-ləri prod-da işlətmə

Növbəti bölmə psql ilə baza səviyyəsində işləmək: \c, \dt, basic select, limit, migration zamanı yoxlamalar və connection refusal səbəbləri.

Bölmə 24. PostgreSQL ilə baza səviyyəsində işləmək

Bu bölmə BA-ya minimal DB əmrlərini verir ki, incident/migration zamanı “DB yaşayırmı?” sualını cavablaya bilsin.

24.1 \l

DB siyahısı:

```
\l
```

24.2 \c

DB seçmək:

```
\c DBF_Content
```

24.3 \dt

Cədvəl siyahısı:

```
\dt
```

Schema-larla:

```
\dt public.*
```

24.4 basic select

Təhlükəsiz select:

```
SELECT * FROM some_table LIMIT 10;
```

24.5 LIMIT ilə təhlükəsiz yoxlama

BA üçün qayda:

- prod-da həmişə LIMIT istifadə edin
- çox böyük table-larda SELECT * etməyin

24.6 update/delete riskləri

UPDATE/DELETE:

- data dəyişir
- audit və rollback çətinləşir

BA üçün: icazəniz olsa belə, prod-da tək etməyin.

24.7 migration zamanı DB-lərin yoxlanması

Migration-dan əvvəl:

- DB connection işləyir?
- disk dolu deyil?
- backup var?

Migration-dan sonra:

- əsas cədvəllər var?
- tətbiq login olur?
- loglarda DB error azalıb?

24.8 owner və permission anlayışı

DB-də də icazə var:

- hansı user hansı schema/table-ı oxuya/yaza bilər

Əgər read-only user ilə belə “permission denied” gəlirsə:

- DBA/DevOps ilə role/grant yoxlanmalıdır.

24.9 connection refusal, pg_hba, listen_addresses əsasları

DB-yə qoşulma alınmırsa:

- DB servisi işləyirmi?
- port dinləyirmi?
- firewall açıqdır?

PostgreSQL tərəfində:

- `listen_addresses` (hansı IP-lərdə dinləyir)
- `pg_hba.conf` (kimə icazə verir)

BA üçün praktik:

- bu faylları dəyişmək BA işi deyil
- ancaq simptomları düzgün toplamaq eskalasiya üçün vacibdir

Növbəti bölmə Postman ilə API yoxlamalarıdır.

Bölmə 25. Postman ilə API yoxlamaları

API problemi olanda BA üçün ən güclü bacarıq: UI-dan asılı olmadan API-nin cavab verib-vermədiyini yoxlamaqdır.

25.1 Postman nədir

Postman:

- HTTP request göndərmək
- response-u (status, headers, body) oxumaq
- auth header-ləri idarə etmək
- collection və environment-lər ilə standart test ssenariləri yaratmaq

25.2 GET request atmaq

Sadə test:

- Method: GET
- URL: `https://docs.company.az/api/health` (placeholder)

Expected:

- 200 OK və sadə JSON/text

25.3 POST request atmaq

POST adətən body tələb edir.

- Method: POST
- Body: JSON

25.4 Headers əlavə etmək

Tipik header-lər:

- Content-Type: `application/json`
- Authorization: `Bearer <token>`
- X-API-KEY: `<key>`

25.5 X-API-KEY nədir

Bəzi internal API-lər Keycloak token əvəzinə X-API-KEY tələb edə bilər.

Təhlükəsizlik:

- API key secret-dir, paylaşılır.

25.6 X-API-KEY ilə request

Postman-də:

- Headers tab → X-API-KEY əlavə edin

CLI alternativ (curl):

```
curl -H "X-API-KEY: <KEY>" https://docs.company.az/api/ping
```

25.7 Authorization header anlayışı

Keycloak tipik “Bearer token” verir:

- Authorization: Bearer eyJ...

Token müddəti bitərsə:

- 401 Unauthorized gələ bilər

25.8 query param və path param

- Query param: ?page=1&size=50
- Path param: /api/items/123

Postman-də:

- Params tab → query param-ları əlavə etmək daha rahatdır.

25.9 body göndərmək

JSON body nümunəsi:

```
{ "from": "2026-01-01", "to": "2026-01-31" }
```

form-data adətən file upload üçün istifadə olunur.

25.10 response oxumaq

3 əsas hissə:

- status code (200/400/401/403/404/500)
- headers (server, content-type, request-id)
- body (error mesajı, data)

BA üçün:

- 500 → server-side error (log lazımdır)
- 401/403 → auth/permission problem
- 404 → route səhv, proxy səhv, və ya feature disabled

25.11 SSL verify problemləri

Internal sertifikatlarda Postman “certificate not trusted” göstərə bilər.

Yanaşma:

- mümkün olduqda root certificate-i sistemə əlavə etmək (Security/DevOps ilə)
- “Disable SSL verification” yalnız diaqnostika üçün, prod intizamı ilə

25.12 internal API endpoint-lərin test edilməsi

Minimal “ping/health” endpoint-ləri saxlayın:

- /health
- /api/ping

Bu endpoint-lər incident zamanı “servis yaşayır” cavabı verir.

25.13 curl-dan Postman-a və Postman-dan curl-a keçid

Postman-də:

- “Code” düyməsi ilə curl export etmək olur

Bu çox faydalıdır:

- DevOps/Dev ilə eyni request-i paylaşmaq
- incident qeydində request-i sənədləşdirmək

Növbəti bölmə auth: Vault/Keycloak axını, servislər down olanda nə baş verir və BA-nın nə qədər bilməli olduğu.

Bölmə 26. Vault, Keycloak və auth əsasları

Auth problemi olanda simptomlar çox müxtəlif görünür: login olmur, token invalid, API 401/403 verir, UI redirect loop-a düşür. Bu bölmə BA üçün “minimum mental model” verir.

26.1 Docs5-də auth axını

Sadə model:

1. User UI-a girir
2. UI user-i Keycloak login-ə yönləndirir
3. User login edir
4. Keycloak token verir
5. UI/API token ilə request-lər edir

Problem harada ola bilər?

- Keycloak down
- Nginx redirect/proxy səhv
- clock/time drift (token time)
- client/realm config dəyişib

26.2 Keycloak nə üçündür

Keycloak:

- istifadəçi identifikasiyası
- rol/permission
- token issuance

BA üçün:

- realm, client, redirect URL anlayışlarını başa düşmək kifayətdir.

26.3 Vault nə üçündür

Vault və ya “vault settings” konsepti:

- secret-ları təhlükəsiz saxlamaq
- tətbiqlərə runtime-da secret təqdim etmək

Bu repo util-lərində “vault.settings.json” mount edilməsi bunu göstərir.

26.4 BA nə qədər bilməlidir

BA üçün məqsəd:

- auth simptomunu düzgün təsvir etmək
- 401 vs 403 fərqi anlamaq
- hansı komponentin down ola biləcəyini təxmin etmək

BA-nın etməməli olduğu:

- secret-ları oxumaq/paylaşmaq
- realm/client konfiqini tək dəyişmək

26.5 connection problemində ilkin yoxlama

Auth problemi gələndə:

- Keycloak URL açılır?
- Nginx 502 verir?
- API “health” işləyir?

CLI yoxlama:

```
curl -I https://keycloak.company.az
```

```
curl -I https://docs.company.az
```

26.6 domain, port, TLS, credentials əlaqəsi

Auth-da 4 qat:

- domain/DNS
- TLS (sertifikat)
- proxy (Nginx)
- credential/token

Səhv qat tapılmadan “password səhvdir” demək tez-tez yanlış olur.

26.7 bu servislər down olanda nə baş verir

- Keycloak down → login olmur, API 401 artar
- Vault/settings yoxdur → app start olmaya bilər və ya DB connection qırılır

BA üçün:

- loglarda `cannot load configuration / vault / secret` kimi sözlərə baxın.

Növbəti bölmə Storage əməliyyat məntiqi: blob/metadata fərqi, storage servis check, preview/file access problemləri.

Bölmə 27. Storage əməliyyat məntiqi

Docs5-də fayllar iki hissə kimi düşünülür:

- metadata (DB-də: fayl adı, ölçü, id, link)
- blob data (storage-da: real binary məzmun)

Bu ayrım storage problemini daha tez tapmağa kömək edir.

27.1 WSS Storage nədir

Storage:

- fayl upload/download üçün servis
- bəzən S3-compatible, bəzən filesystem-based ola bilər

27.2 blob anlayışı

Blob — “binary large object”: PDF, DOCX, şəkil, arxiv və s.

27.3 storage metadata və blob data fərqi

Simptom nümunələri:

- UI faylı “görür”, amma download alınmır → metadata var, blob yoxdur/əlçatmazdır
- fayl ümumiyyətlə görünmür → metadata DB-də yoxdur və ya query problemi var

27.4 storage servisin ayaqda olub-olmadığını yoxlamaq

Yanaşma:

- container isə: `docker ps, docker logs`
- host servis isə: `systemctl status`

Endpoint varsa:

```
curl -I http://storage.internal/health
```

27.5 storage config faylları

Storage config yerləşimi mühitə görə dəyişir.

BA üçün praktik:

- config yolunu tapmaq üçün compose volume-larına baxın
- loglardan config path çıxara bilərsiniz

27.6 migration kontekstində storage

Storage migration ssenariləri:

- köhnə storage-dan yeniyə keçid
- path/namespace dəyişməsi

Bu mərhələdə:

- preview generator util-ləri
- file link update util-ləri

işlədilə bilər.

27.7 preview və file access problemləri

Preview açılırsa:

- storage əlçatandır?
- preview generation servisi işləyir?
- fayl tipi dəstəklənirmi?

Triage:

- app loglarda preview error-a bax

- storage loglarda access denied/not found var?

27.8 storage ilə bağlı tipik səhvlər

- AccessDenied / 403 → credential/permission
 - NotFound → blob yoxdur, path səhv
 - Timeout → network/firewall
-

Növbəti bölmə portlar və firewall qaydalarıdır: komponentlər arasındakı şəbəkə icazələrinin düzgün planlanması.

Bölmə 28. Portlar və firewall qaydaları (hansı server → hansı server → hansı port)

Bu bölmə Docs5 arxitekturasında şəbəkə icazələrini (firewall rules) düzgün planlamaq üçündür: hansı komponent hansı komponentə hansı portdan çıxmalıdır.

Vacib qeyd (dəqiqlik): Bu workspace-də Sources/ altında gördüyümüz docker-compose.yml faylları əsasən utility/migration konteynerləridir və “tam prod stack” (nginx, docs web/api, keycloak, storage və s.) compose-u burada yoxdur. Ona görə aşağıdakı port cədvəli Docs5 üçün tipik/baza port matrisi kimi verilir və sonda mühitinizdən real port siyahısını 5 dəqiqəyə çıxarma addımları əlavə olunur.

28.1 Port anlayışı: inbound vs outbound

- Inbound: digər serverdən bu serverə gələn trafik (firewall-da “allow incoming”).
- Outbound: bu serverin başqa serverə getməsi (çox mühitdə outbound açıq olur, amma korporativ şəbəkələrdə outbound da məhdud ola bilər).

Əsas prinsip:

- “Həmişə açıq” yox, yalnız lazım olan mənbədən (source IP/subnet) yalnız lazım olan porta icazə.

28.2 Tipik Docs5 komponentləri və portları

Aşağıdakı portlar Docs5 tipli arxitekturalarda ən çox rast gəlinən standartlardır:

- Reverse proxy (Nginx): 80/tcp, 443/tcp
- PostgreSQL: 5432/tcp
- Keycloak (HTTP): 8080/tcp (bəzi qurulumlarda)
- Keycloak (HTTPS): 8443/tcp (bəzi qurulumlarda)
- Vault: 8200/tcp
- Storage (S3-compatible): çox vaxt 9000/tcp (MinIO default), bəzən 443/tcp (əgər storage front-da HTTPS-dirsə)
- Docs5 Web/API (Kestrel və ya app server): çox vaxt 5000/tcp, 5001/tcp (mühitə görə dəyişir)

Sizdə fərqli ola bilər. Dəqiq portlar üçün 28.6 bölməsindəki “real çıxartma” addımlarını edin.

28.3 Arxitektura variantları (port ehtiyacı buna görə dəyişir)

Variant A: “Hər şey 1 serverdə” (ən sadə)

- Internet/LB → Nginx server: 80/443
- Nginx → Docs5 app: localhost (host daxilində) → firewall lazım olmur
- App → PostgreSQL: localhost → firewall lazım olmur
- App → Keycloak/Vault/Storage: yenə localhost və ya eyni host network

Bu variantda firewall ən çox:

- 22/tcp (SSH) yalnız jump/office IP-dən
- 80/443 internet/LB-dən

açmaqla kifayətlənir.

Variant B: “Servislər ayrı serverlərdə” (ən real prod)

Bu halda firewall rules vacib olur:

- Nginx server → App server (internal port)
- App server → DB server 5432
- App server → Keycloak 8080/8443
- App server → Vault 8200

- App server → Storage 9000 və ya 443

28.4 Tam port matrisi (serverdən serverə)

Aşağıdakı cədvəl “split deployment” üçün minimum icazələri göstərir.
<SUBNET_APP>, <SUBNET_DMZ> kimi yerləri sizin şəbəkəyə uyğunlaşdırın.

Mənbə (source)	Hədəf (destination)	Port	Nə üçün
User Internet / Corporate Users	Nginx (DMZ)	80/tcp	HTTP (redirect üçün)
User Internet / Corporate Users	Nginx (DMZ)	443/tcp	HTTPS əsas trafik
Jump/Office Admin IP	Bütün serverlər	22/tcp	SSH admin giriş
Nginx (DMZ)	Docs5 App (APP subnet)	5000/tcp və ya tətbiq portu	Reverse proxy → upstream
Docs5 App (APP subnet)	PostgreSQL (DB subnet)	5432/tcp	DB connection
Docs5 App (APP subnet)	Keycloak (AUTH subnet)	8080/tcp və ya 8443/tcp	Token/auth əməliyyatları
Docs5 App (APP subnet)	Vault (SEC subnet)	8200/tcp	Secret/config oxuma

Mənbə (source)	Hədəf (destination)	Port	Nə üçün
Docs5 App (APP subnet)	Storage (STORAGE subnet)	9000/tcp və ya 443/tcp	Fayl upload/ download
Preview/ Migration util serverləri (əgər ayrıca işləyirsə)	Eyni hədəflər	eyni portlar	util-lər də DB/ storage/auth-a çıxır

Qeyd: App server portu (cədvəldə 5000) tam sizin Kestrel/compose konfiqinizdən asılıdır. Bəzi qurulumlarda Nginx app-ə 127.0.0.1:5000 ilə gedir (eyni host), bəzi qurulumlarda ayrı hosta gedir.

28.5 UFW (və ya iptables) səviyyəsində nümunə qaydalar

Bu hissə konseptual nümunədir (placeholder IP-lərlə). Prod-da dəyişiklik yalnız approval/proseslə.

28.5.1 Nginx server (DMZ) üçün

- Incoming:
 - allow 80/tcp, 443/tcp (internet/LB source-lardan)
 - allow 22/tcp (yalnız jump/office IP)

- Outgoing:
 - App serverə upstream port (məs: 5000/tcp)

Nümunə:

```
sudo ufw allow from <OFFICE_IP>/32 to any port 22 proto tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
# upstream üçün outbound adətən açıq olur; məhdud mühitdə ayrıca
allow lazım ola bilər.
```

28.5.2 App server üçün

- Incoming:
 - upstream port (5000/5001 və ya sizin port) yalnız Nginx server IP-dən
 - 22/tcp yalnız admin IP-dən
- Outgoing:
 - DB 5432
 - Keycloak 8080/8443
 - Vault 8200
 - Storage 9000/443

Nümunə:

```
sudo ufw allow from <NGINX_IP>/32 to any port 5000 proto tcp
sudo ufw allow from <OFFICE_IP>/32 to any port 22 proto tcp
```

28.5.3 DB server üçün

- Incoming:
 - 5432/tcp yalnız App server IP/subnet-dən
 - 22/tcp yalnız admin IP

```
sudo ufw allow from <APP_SUBNET> to any port 5432 proto tcp
```

```
sudo ufw allow from <OFFICE_IP>/32 to any port 22 proto tcp
```

DB portunu internetə açmaq olmaz. pgAdmin üçün “internetdən 5432 açmaq” əvəzinə SSH tunnel istifadə edin (Bölmə 23).

28.6 Sizin mühit üçün “tam və dəqiq” port siyahısını çıxartmaq (5 dəqiqəlik prosedur)

Aşağıdakı addımların məqsədi: “bizim arxitektura” üzrə real portları (5000 yerinə bəlkə 7000, 9000 yerinə 443 və s.) dəqiq tapmaq.

Addım 1: hər serverdə dinləyən portları çıxart

```
sudo ss -tulnp
```

Fokus olun:

- :80, :443 (nginx)
- :5432 (postgres)
- :8080, :8443 (keycloak)
- :8200 (vault)

- :9000 (minio/storage)
- app portları (:5000, :5001 və s.)

Addım 2: Docker konteyner port publish-lərini yoxla (əgər container-dirşə)

```
docker ps --format 'table {{.Names}}\t{{.Ports}}'
```

Buradan görəcəksiniz:

- host port → container port mapping

Addım 3: Nginx upstream portunu dəqiq tap

Nginx config-də proxy_pass sətrinə baxın:

- proxy_pass http://127.0.0.1:5000;
- və ya proxy_pass http://10.0.10.25:7000;

Axtarış:

```
sudo grep -R "proxy_pass" /etc/nginx -n
```

Addım 4: App konfiqində DB host/port-a bax

App appsettings.json və ya env-də DB host/port olur. Bu workspace-də util-lərin appsettings-lərində port 5432 görünür.

Tətbiq serverində:

```
grep -R "Port=5432" -n /opt/wss 2>/dev/null
```

Addım 5: Çıxan “real portlar” ilə cədvəli yenilə

- Cədvəldəki “5000/5001” kimi yerləri sizin real portlarla əvəz edin.
- Hər rule üçün source IP/subnet-i məhdudlaşdırın.

28.7 BA üçün praktiki qayda

- DB portu (5432) internetə açılmır — yalnız app subnet/jump ilə.
- “İşləmir” deyəndə ilk sual: port dinləyir? (`ss -tulpn`) və firewall bloklayır? (`ufw status verbose`).